Capteur Meteo Wi-fi

Joao Marques Chaves (joao.marques@eduvaud.ch) Numéro de projet : **2002**

Date de début : **05.08.2024**
Date de fin : **10.09.2024**
Durée : **05 semaines**
Type de projet : **Travail de diplôme**
Responsable : **M. Bovey**

Table des matières

1 Acronymes **6**

2 Cahier des charges **7**
2.1 Introduction . . . 7
2.2 Principe de fonctionnement . . . 7

3 Pré-étude **8**
3.1 Introduction . . . 8
3.2 Planning . . . 8
3.3 Schéma bloc . . . 9
3.4 Descpritions des blocs détaillées . . . 9
3.4.1 Bloc Convertisseur USB-UART . . . 9
3.4.2 Bloc Module Wi-fi . . . 9
3.4.3 Bloc Capteur Temp/Hum/Press/C02 . . . 10
3.4.4 Bloc RTC . . . 10
3.4.5 Bloc LED . . . 10
3.4.6 Bloc Microcontrôleur . . . 11
3.4.7 Bloc USB-C . . . 11
3.4.8 Bloc Contrôleur de charge . . . 11
3.4.9 Bloc Régulateur 3.3V . . . 12
3.4.10 Bloc Pile . . . 12
3.5 Consommation global . . . 12
3.6 Intégration mécanique . . . 12
3.7 Estimation des coûts en CHF . . . 13
3.8 Faisabilité du projet . . . 13

4 Design éléctronique **14**
4.1 Introduction . . . 14
4.2 Système d'alimentation . . . 14
4.2.1 Choix principal . . . 14
4.2.2 Dimensionnement . . . 14
4.2.3 Schéma . . . 14
4.3 USB-C . . . 15
4.3.1 Choix principal . . . 15
4.3.2 Dimensionnement . . . 15
4.3.3 Schéma . . . 16
4.4 Piles/Accus . . . 17
4.4.1 Choix principal . . . 17
4.4.2 Dimensionnement . . . 17
4.4.3 Schéma . . . 18
4.5 LED RGB . . . 19
4.5.1 Choix principal . . . 19
4.5.2 Dimensionnement . . . 19
4.5.3 Schéma . . . 19
4.6 Capteur . . . 20
4.6.1 Choix principal . . . 20
4.6.2 Dimensionnement . . . 20
4.6.3 Schéma . . . 20
4.7 RTC . . . 21
4.7.1 Choix principal . . . 21
4.7.2 Dimensionnement . . . 21
4.7.3 Schéma . . . 21
4.8 Module Wifi - ESP32 . . . 22
4.8.1 Choix principal . . . 22
4.8.2 Dimensionnement . . . 22
4.8.3 Schéma . . . 24
4.9 Microcontrôleur . . . 25
4.9.1 Choix principal . . . 25

4.9.2 Dimensionnement . . . 25
4.9.3 Schéma . . . 25

5 Design circuit imprimé **26**
5.1 Introduction . . . 26
5.2 Caractéristiques du circuit . . . 26
5.3 Placement et routage des composants . . . 27

6 Programmation de l'ESP32 **29**
6.1 Introduction . . . 29
6.2 Prérequis . . . 29
6.3 Schéma . . . 29
6.4 Marche à suivre . . . 30
6.4.1 Mettre l'esp en mode download . . . 30
6.4.2 Programmation de l'ESP32 . . . 31
6.4.3 Vérification de la programmation . . . 32

7 Montage et tests **33**
7.1 Test BMe280, ENS160, MCP74911 . . . 33
7.2 Montage . . . 33
7.3 Amélioration . . . 33

8 Firmware **34**
8.1 Introduction . . . 34
8.2 Configuration Harmony . . . 34
8.2.1 Configuration des périphériques . . . 35
8.3 Librairies utilisées . . . 36
8.4 BME280 . . . 37
8.5 ENS160 . . . 37
8.5.1 ENS160Init . . . 38
8.5.2 ENS160ReadDataAqi . . . 39
8.5.3 ENS160ReadDataTvoc et ENS160ReadDataEco2 . . . 39
8.5.4 SwapMsbLsb . . . 40
8.6 ESP32 . . . 41
8.6.1 SendATCommand . . . 41
8.6.2 ESP32Init . . . 42
8.6.3 ESP32isStarted . . . 43
8.6.4 ESP32GetIpAndMac . . . 43
8.6.5 ESP32ConnectWifi . . . 44
8.6.6 ESP32ChangeMode . . . 45
8.6.7 PublishMQTT . . . 45
8.6.8 SetupMQTTAndPublish . . . 46
8.6.9 ESP32DisconnectTCPServer . . . 46
8.6.10 ESP32SendTCP . . . 47
8.7 Gestion menu . . . 48
8.7.1 Menu principal . . . 49
8.8 Sauvegarde . . . 49
8.9 RTC . . . 50
8.10 Gestion LED RGB . . . 50
8.11 Gestion alimentation . . . 50

9 Software **51**
9.1 Introduction . . . 51
9.2 MQTT Broker . . . 51
9.2.1 Mosquitto . . . 51
9.3 TCP . . . 52

10 Mesures **53**
10.1 Matériel utilisé . . . 53
10.2 I2C . . . 53
10.2.1 BME280 . . . 53
10.2.2 ENS160 . . . 54
10.2.3 MCP74911 . . . 55
10.3 Uart . . . 56
10.3.1 ESP32 . . . 56
10.3.2 FT230X . . . 57
10.4 Alimentation . . . 58
10.5 Lecture ADC . . . 58

11 Test mémoire Flash **59**

12 Auto-analyse **60**
12.1 Acquis du TD . . . 60
12.2 Prise en compte de l'environnement social . . . 60
12.3 Prise en compte de l'environnement naturel . . . 60
12.4 Leadership et développement personnel . . . 60

13 Conclusion **61**

14 bibliographie **62**

15 Annexe **63**
15.1 Cahier des charges . . . 63
15.2 Planification . . . 63
15.3 Journal de travail . . . 63
15.4 Procès-verbaux des séances hebdomadaires . . . 63
15.5 Schémas électroniques . . . 63
15.6 Fichiers de fabrication . . . 63
15.7 Code source firmware . . . 63

1 Acronymes

- **NiMH** : Nickel-Metal Hybride
- **MQTT** : Message Queuing Telemetry Transport
- **RTC** : Real Time Clock
- **MCU** : Microcontrôleur
- **PCB** : Printed Circuit Board
- **UFP** : Upstream Facing Port
- **DFP** : Downstream Facing Port
- **DRP** : Dual-Role Port

2 Cahier des charges

2.1 Introduction

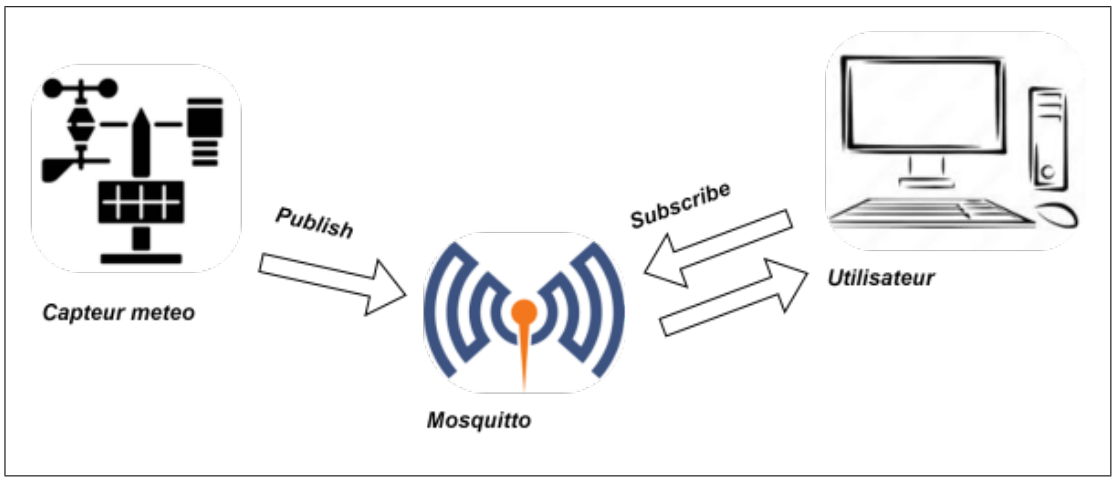
Le but de ce projet de diplôme est de réaliser un capteur météo capable des mesures suivantes : la pression, l'humidité, la température et la qualité de l'air. Pour cela, les mesures seront envoyées par Wi-Fi grâce à un module ESP32 avec des prévisions météorologiques. Alimenté principalement par USB-C, le dispositif disposera également d'une alimentation de secours par batterie (pile ou accu NiMH) en cas de coupure d'alimentation. Le prototypage repose sur l'ancienne B et A, qui doit être réduite pour tester le nouveau design, supprimer les bugs du firmware et implémenter de nouvelles fonctionnalités. Il sera nécessaire de tester l'ancien capteur pour lister les bugs présents dans le firmware et les améliorations nécessaires à la version C.

Voir annexe ? pour voir le cahier de charges détaillant les exigences précises du projet .

2.2 Principe de fonctionnement

Le système sera alimenté par USB-C et également des piles ou accu seront en cas de coupure. Le système aura deux capteurs qui seront capables de faire des mesures de pression, humidité, température et de la qualité de l'air.

Ensuite, avec ces différentes mesures, le but sera de les envoyer sur une page Web grâce à un module ESP32. Pour ce faire, il faudra dans un premier temps envoyer ces mesures à un mqtt broker (par exemple : Mosquitto) et ensuite nous viendrons chercher ces informations sur le broker avec un système de publish et sub.



La page Web utilisée par l'utilisateur pourrait être configurée pour recevoir différentes informations via un broker MQTT. Un broker MQTT est un point de communication central qui reçoit des messages de la part des éditeurs (publishers) et les distribue aux abonnés (subscribers) de ces messages.

Par exemple, Monsieur X envoie au broker l'information qu'il vend un livre en publiant un message sur un topic spécifique, tel que "vente/livre". Madame Y, de son côté, a indiqué au broker qu'elle voulait recevoir des annonces sur des livres d'occasion en s'abonnant à ce même topic "vente/livre". Le broker, jouant son rôle de distributeur central, détecte que Monsieur X a publié un message sur ce topic et que Madame Y est abonnée à celui-ci. Il va donc automatiquement transmettre le message de vente de Monsieur X à Madame Y.

3 Pré-étude

3.1 Introduction

Cette phase de pré-étude a pour but d'analyser en détail les composants essentiels du projet afin de bien cerner les contraintes et exigences. Cela permet également de concevoir un concept viable et optimisé du produit, en identifiant et éliminant les obstacles potentiels avant la phase de conception. Ainsi, cette approche garantit l'obtention d'un produit final opérationnel. De plus, avant le début des recherches sur les différents composants utiles pour ce projet, j'ai effectué des tests avec le prototype existant pour pouvoir prendre note de ce qui est fonctionnel ou non.

3.2 Planning

Après avoir pris connaissance du cahier des charges, j'ai pu préparer un planning pour organiser efficacement le temps alloué au développement de ce projet.

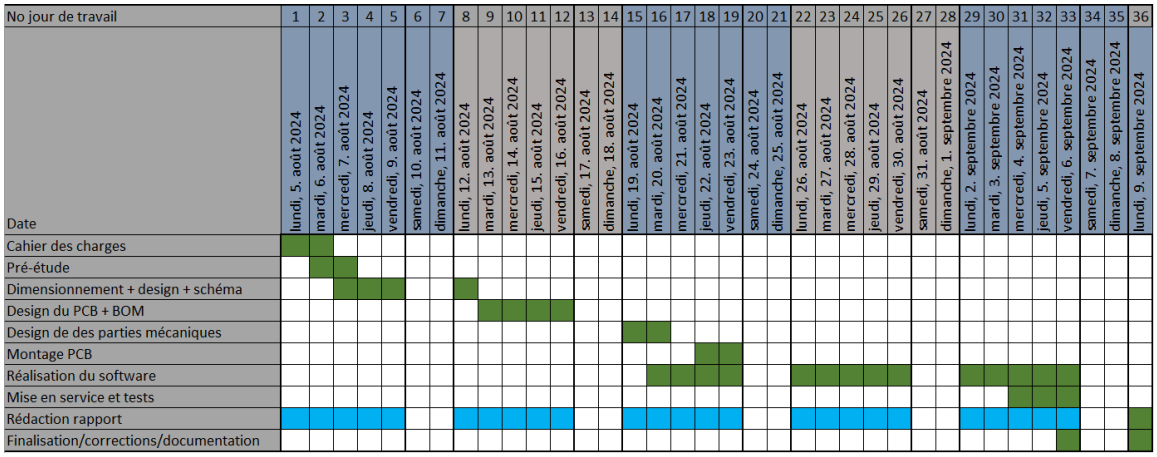


FIGURE 1 – Planning

La planification finale sera incluse en annexe ?, avec les tâches effectivement réalisées en gras. Vous pouvez aussi retrouver le journal de travail en annexe ?.

Durant toute la phase de ce diplôme, des réunions hebdomadaires en étaient réalisées avec le maître de diplômes afin de voir l'évolution du projet. Lors de ces réunions, des procès-verbaux ont été réalisés pour chaque séance, que vous pouvez retrouver en annexe ?.

3.3 Schéma bloc

Le schéma bloc illustré dans la Figure 2 met en évidence les composants présents sur la nouvel version du prototype, on pourra voir les ajout effectuer sur cette version C.

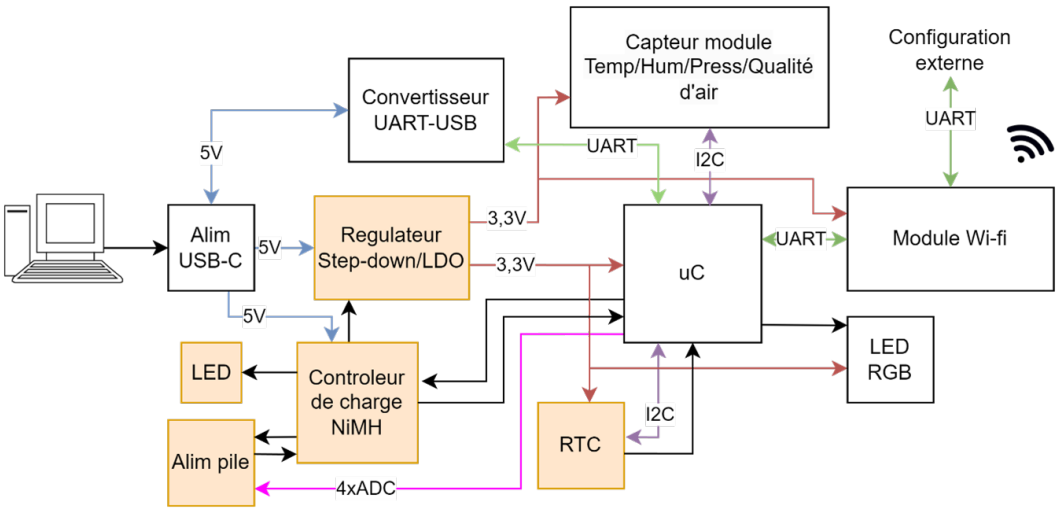


FIGURE 2 – Schéma bloc

3.4 Descpritions des blocs détaillées

Dans cette section, chaque bloc du schéma est décrit de sorte à comprendre quelles sont les contraintes et quelles est leur utillité.

3.4.1 Bloc Convertisseur USB-UART

Pour la conversion USB-UART, j'ai décidée de reprendre le **FT230X** [1] vu qu'il est fonctionnel sur le prototype. Ce composant me permettra de faire le pont entre la communication USB du PC et UART du microcontrôleur.

- Caractéristiques :
  - Tension d'alimentation de 5V où 3.3V
  - Vitesse de transfère de données de 300 baud à 3 Mbaud
  - Normes : USB à RS232/RS422/RS485
  - Buffer de transmission de 512 byte
  - Mémoire interne programme de 2048 byte
  - Consommation : 8 mA

3.4.2 Bloc Module Wi-fi

Pour le module Wi-Fi, je vais conserver le même module ESP32, mais pour gagner de la place sur le PCB, j'utiliserais uniquement **l'ESP32-C3-WROOM-02** [2] et non la version Click Board de chez MIKROE.

- Caractéristiques :
  - Tension d'alimentation de 3.3V
  - Modes de fonctionnements : AP (point d'accès), mode client Wi-fi
  - Protocoles réseau : IPv4, TCP, UDP, HTTP, FTP
  - Communication UART
  - Consommation : 355 mA pour TX et 95 mA pour RX, 240 uA en light-sleep

1. Fiche technique du FT230X
2. Fiche technique de l'ESP32-C3-WROOM-02

3.4.3 Bloc Capteur Temp/Hum/Press/C02

Pour les capteurs, je vais conserver le **BME280** [3] pour les mesures de température, d'humidité et de la pression. Et ajouter le capteur **ENS160** [4] pour mesurer la qualité de l'air (CO2). Pour simplifier le routage et le montage, je vais utiliser le module **SEN-22858** de SparkFun, qui combine ces capteurs sur un petit circuit que je pourrai brancher sur mon PCB.

- Caractéristiques BME280 :
  - Tension d'alimentation de 3.3V
  - Communication SPI ou I2C
  - Operating Range : -40 à 85°c, 0 à 100%, 30000 à 110000Pa
  - Resolution : 0.01°C, 0.008%, 0.18Pa
  - Consommation durant la mesure : Humimidy 340uA, Pressure 714uA, Temperature 350uA

- Caractéristiques ENS160 :
  - Tension d'alimentation de 3.3V
  - Communication SPI ou I2C
  - Range : 1 à 5, 400 à 65000ppm, 0 à 65000ppb
  - Resolution : 1, 1ppm, 1ppb
  - Consommation : DeepSleep 0.01mA, IDLE 2.5mA, Standard 29 mA

3.4.4 Bloc RTC

Pour la RTC, j'ai décidé de reprendre le **MCP79411** [5] qui est utilisé dans les kits PIC32 [6].

- Caractéristiques :
  - Tension d'alimentation entre 1.8 est 5.5V
  - Communication I2C
  - Consommation SRAM/RTCC : 300uA pour la lecture et 400uA pour l'écriture
  - Consommation EEPROM : 400uA pour la lecture et 3mA pour l'écriture

3.4.5 Bloc LED

Pour les différentes indications lumineuses du système, une LED RGB est présente, j'ai décidé de garder le même modèle que la dernière version, qui est le **APHF1608LSEEQBDZGKC** [7] de chez Kingbright.

- Caractéristiques :
  - Tension d'alimentation LED rouge 2.1V
  - Tension d'alimentation LED verte 3.1V
  - Tension d'alimentation LED bleu 3.1V
  - Consommation : 2mA

Une LED rouge est également là pour le contrôleur de charge, le modèle **SML-P11VTT86R** [8] a été choisit pour son faible courant.

- Caractéristiques :
  - Tension d'alimentation : 1.8V
  - Consommation : 1mA

3. Fiche technique du BME280
4. Fiche techniques du ENS160
5. Fiche technique du MCP79411
6. Carte de développement sur pic32 réaliser par l'ETML-ES pour les cours de MINF.
7. Fiche technique du APHF1608LSEEQBDZGKC
8. Fiche technique du SML-P11VTT86R

3.4.6 Bloc Microcontrôleur

Pour le microcontrôleur, j'ai décidé de garder la même modèle que la dernière version, qui est le **PIC32MX130F256** [9].

- Caractéristiques :
  - Tension d'alimentation entre 2.3 et 3.6V
  - UART disponible 2
  - I2C disponible 2
  - ADC disponible 10-13
  - OC disponible 5
  - I/O disponible 21 ou 35
  - Consommation : 30mA

La Table 1 répertorie les diverses configurations de signaux qui pourraient être employées dans le design final afin de se faire une première idée de la configuration minimale du MCU.

Système à commander	**Nombre d'I/O**	**Type d'I/O**
Capteur	2	I2C
RTC	2	I2C
Module Wi-Fi	2-4	UART
Convertisseur UART-USB	2-4	UART
LED RGB	3	OC
Alim pile	4	ADC
Autres	8	GPIO

Table 1 – Signaux MCU

3.4.7 Bloc USB-C

Le système sera principalement alimenté par **USB-C**, qui est connecté à l'ordinateur de l'utilisateur. L'USB-C servira également à communiquer avec le microcontrôleur via le convertisseur USB-UART.

- Caractéristiques :
  - Tension d'alimentation de 5V
  - Communication USB
  - Consommation soutirable : 500mA si conecter en USB type 2 ou 900mA en USB type 3

3.4.8 Bloc Contrôleur de charge

Pour le contrôleur de charge, je suis partie sur le modèle **BQ25172** [10] de chez Texas Instruments. Il permet de charger des piles NiMH et peut gérer jusqu'à 6 cellules, et il a également la possibilité de mettre une thermistance pour contrôler si les piles chauffent ou non.

- Caractéristiques :
  - Tension d'alimentation entre 3 et 18V
  - Tension de sortie maximum de 10.5V
  - Courant de charges 10 mA a 800mA
  - Nombre de cellule : 1-6
  - Type de cellule : NiMH

9. Fiche technique du PIC32MX130F256
10. Fiche technique du BQ25172

3.4.9 Bloc Régulateur 3.3V

Le régulateur choisi est le modèle **173950336** [11] de Wurth Elektronik. Il s'agit d'une alimentation à découpage intégrée dans un boîtier.

- Caractéristiques :
  - alimentation step down
  - Tension d'entrée entre 4,75V et 36V
  - Tension de sotie de 3.3V
  - Consommation soutirable : 500mA

3.4.10 Bloc Pile

Pour les piles, je suis partie pour des piles AAA et il y'aura 4 piles pour garantir une tension d'alimentation de 4,8 V. Les piles **Alcaline** [12] ont une tension typique de 1,5 V alors que les piles **NiMH** [13] ont une tension typique de 1,2 V. Les piles seront mesurées à l'aide de 4 entrées ADC du microcontrôleur pour pouvoir distinguer si le système est sous pile Alcaline ou NimH.

3.5 Consommation global

Avec la Table 2 on peut voir ce dont le système a besoin dans le cas maximum de 420mA. Cela montre que le régulateur avec ces 500mA peut supporter le pire cas de consommation.

Composant	**Max**
Capteur	29mA
RTC	3mA
Module Wi-Fi	355mA
LED RGB	2mA
LED	1mA
Microcontrôleur	30mA
Total	420mA

Table 2 – Table des consommations électrique 3.3V

3.6 Intégration mécanique

Une mise en boîtier m'est demandée. J'ai décidé de le réaliser en impression 3D en PLA au lieu d'acheter un boîtier existant et de devoir le modifier. Mes connaissances en conception me permettent de créer sans trop de complexité ce type de boîtier, et cela facilitera également l'intégration du PCB, des piles AAA et du connecteur USB-C.

11. Fiche technique du 173950336
12. Fiche technique d'une pile Alcaline
13. Fiche technique d'une pile NiMH

3.7 Estimation des coûts en CHF

Ci-dessous, avec la Table 3 vous pouvez voir une approximation des dépenses dans la conception de ce projet. On peut voir que nous obtenons un prix d'environ 124.-.

Composant	**Max**
Régulateur 3.3V	6.-
Convertisseur UART-USB	2.-
Contrôleurs de charges	2.-
Module Wi-Fi	2.-
Capteur	30 .-
RTC	1.-
LED RGB	1.-
LED	1.-
Microcontrôleur	4.-
Autres composants	20.-
PCB	50.-
Boîtier	5.-
Total	124.-

Table 3 – Table de prix

3.8 Faisabilité du projet

Selon les informations recueillies, le projet est tout à fait réalisable dans son intégralité. Cependant, des questions subsistent concernant la partie logicielle, plus particulièrement le broker MQTT que je ne connais pas. Une recherche doit être effectuée pendant le temps d'attente des commandes (PCB, composants).

4 Design éléctronique

4.1 Introduction

Dans cette section, nous allons examiner en détail les différentes étapes qui composent la conception du système. Cela englobe la sélection des composants électroniques, la phase de dimensionnement et la création des schémas.

4.2 Système d'alimentation

Le montage sera principalement alimenté par le port USB-C, qui fournira une sortie de 5V DC. En cas de coupure, les piles ou accus prendront le relais avec une tension minimale de 4,8V et maximale de 6V. Le système, quant à lui, sera principalement alimenté par un régulateur de 3,3V.

4.2.1 Choix principal

Choix du régulateur de tension 3.3V :
Le régulateur choisi est le modèle 173950336 de Wurth Elektronik. Il s'agit d'une alimentation à découpage intégrée dans un boîtier. Son courant de sortie peut atteindre 0,5A avec une tension de sortie de 3,3V. Il peut accepter une tension d'entrée comprise entre 4,75V et 36V(page 4 du datasheet [14]).

4.2.2 Dimensionnement

Aucun dimensionnement particulier n'a été nécessaire. Toutes les informations concernant les condensateurs de découplage sont disponibles dans les documentations techniques du composant. Il est recommandé, selon le schéma de principe, d'utiliser un condensateur de 10 µF pour l'entrée et de 22 µF pour la sortie (page 34-35 du datasheet).

4.2.3 Schéma

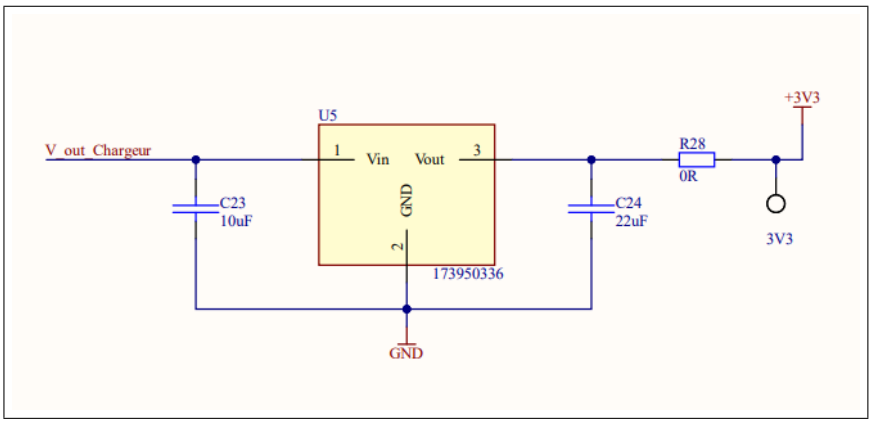


FIGURE 3 – Schéma du régulateur 3.3V

14. Fiche technique du 173950336

4.3 USB-C

Le système sera principalement alimenté par l'USB-C et permettra également la communication entre un PC et le système via un convertisseur USB-UART.

4.3.1 Choix principal

USB-C :
Pour pouvoir effectuer le design et faire les recherche dans les fiche technique il me faut connaître la configuration de notre port USB-C.

Il existe trois type de configurations pour l'USB-C :

- DFP : il s'agit du port d'un appareil qui alimente et transmet des données vers d'autres appareils. C'est celui qui donne, pas celui qui reçoit.

- UFP : port d'un appareil qui est alimenté et reçoit des données d'un DFP. Par exemple, un port USB-C sur un disque dur externe ou un écran serait généralement un UFP car il reçoit des données et est alimenté par l'appareil hôte (DFP).

- DRP : Certains ports USB-C sont capables de fonctionner à la fois comme un DFP ou un UFP, selon les besoins des appareils connectés. Ces ports sont appelés ports à double rôle (DRP) et sont conçus pour être polyvalents, s'adaptant à différentes situations de connexion.

Pour notre système, j'aurais besoin d'un port de type UFP. Vu que le système doit être capable d'être alimenté par le PC mais doit aussi être capable de transmettre des données.

Convertisseur USB-UART :
En ce qui concerne le convertisseur USB-UART, j'ai décidé de conserver le modèle FT230X qui était déjà utilisé dans l'ancienne version. Cela a permis de reprendre en grande partie le schéma de l'ancienne version.

4.3.2 Dimensionnement

USB-C :
Aucun dimensionnement particulier n'a été nécessaire. Les informations utiles se trouvent dans la fiche technique . Pour utiliser l'USB-C en mode UFP, je dois placer des résistances de pull-down de 5,1 kohm sur les broches CC1 et CC2.

Convertisseur USB-UART :
Aucun dimensionnement particulier n'a été nécessaire. Toutes les informations concernant les condensateurs et résistances sont disponibles dans la fiche technique.

Dans la fiche technique, nous pouvons trouver un schéma indiquant les différentes valeurs des condensateurs et des résistances sur les broches USBDM (9) et USBDP (8). On y trouve également la valeur de l'inductance recommandée, qui est le modèle MI0805K601R-10, ainsi que les valeurs des résistances de pull-up pour les sorties UART, qui sont de 10 kohm.

4.3.3 Schéma

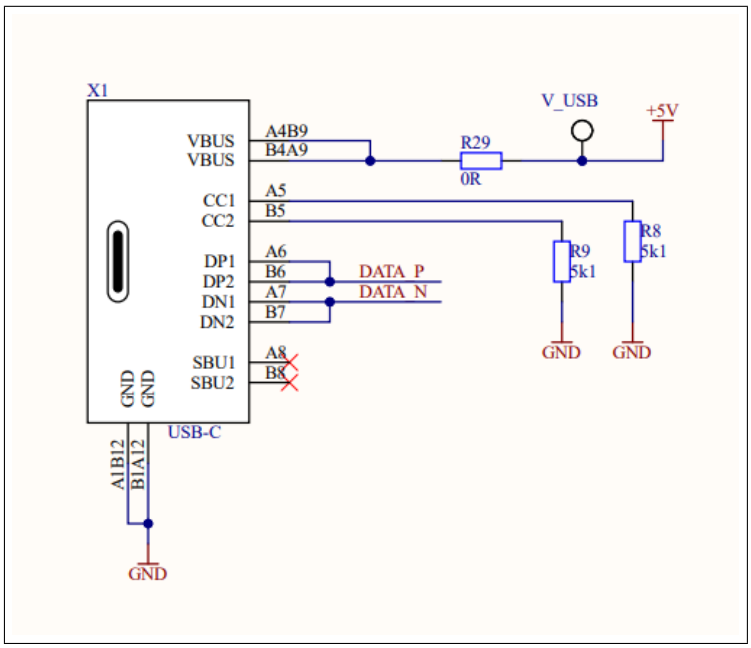


FIGURE 4 – Schéma du connecteur USB-C

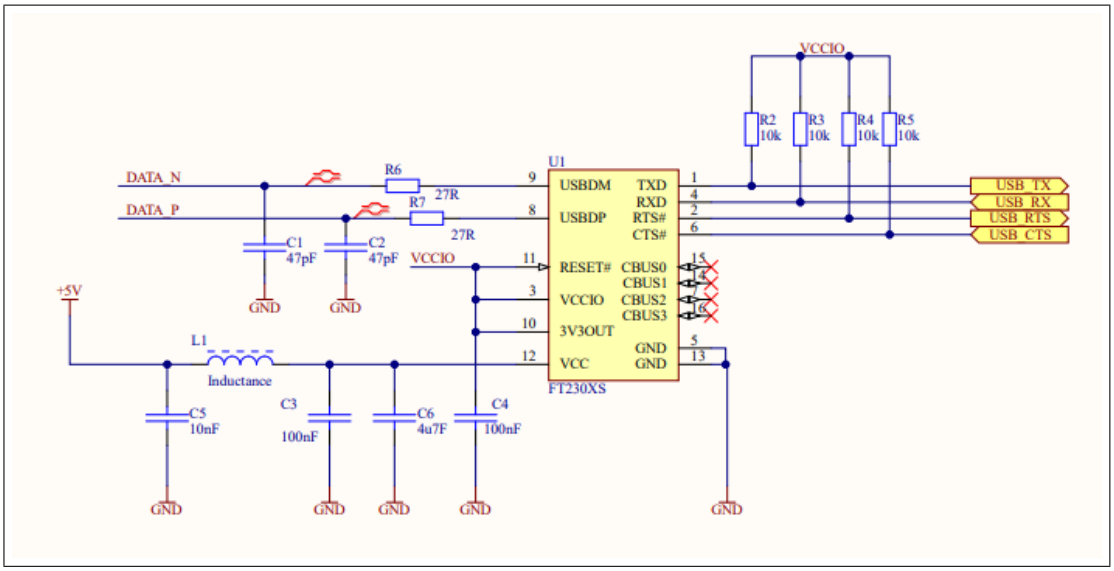


FIGURE 5 – Schéma du convertisseur USB-UART

4.4 Piles/Accus

Les piles et accus sont là pour alimenter le système en cas de coupure d'alimentation. Un système de charge est également mis en place : lorsque le système est alimenté par l'USB-C, les accus se rechargent. De plus, un dispositif de contrôle vérifie si les piles sont rechargeables ou non. Si ce n'est pas le cas, la charge ne doit pas s'effectuer.

4.4.1 Choix principal

piles/Accus :
Pour la réalisation de ce projet, j'ai une contrainte de taille pour le PCB qui doit mesurer 6 x 6 cm. Pour cela, j'ai décidé d'utiliser des piles AAA, car des piles AA dépasseraient la taille disponible sur le PCB.

Contrôleur de charge :
Pour la charge, j'ai choisi le modèle BQ25172 de Texas Instruments. Il s'agit d'un système intégré qui gère les piles NiMH. De plus le BQ25172 [15] a une pin qui permet de savoir si le système est en charge ou non et également s'il y a un problème.

Commande mosfet :
Pour gérer la présence de piles rechargeables ou non, j'ai décidé de mettre en place une commande MOSFET qui, via le microcontrôleur, permettra de déconnecter les piles/accus de la tension de charge. Pour cela, nous avons besoin d'un MOSFET à canal P, avec un VGS(th) d'au moins 3V. En effet, comme la source du MOSFET sera connectée aux piles, nous aurons une tension de 6V dans le cas de piles non rechargeables, tandis que la tension sur la gate sera de 3,3V, ce qui crée une différence de potentiel de 6V - 3,3V = 2,7V.

Pour cela j'ai choisit de prendre le modèle **SI5618-TP** [16] de Micro Commercial Components qui a une tension VGS(th) de 3V pour un courant maximum de 1.9 A.

4.4.2 Dimensionnement

Contrôleur de charge :
Pour les condensateurs à l'entrée et à la sortie, les valeurs recommandées sont de 1 µF pour Cin (C21) et Cout (C22).

Ensuite, pour la sortie 3 (TS), nous utilisons une NTC. La fiche technique recommande une valeur de 10 kohm et suggère le modèle 103AT-2.

Sur la sortie 5 (STAT), j'ai une LED à faible consommation (1 mA) pour indiquer l'état. La LED nécessite un courant de 1 mA, une tension de 1,8V, et notre tension d'alimentation est de 5V.
Calcul :
Pour dimensionner la résistance, j'utilise la loi d'Ohm.

$$R_{20} = \frac{+5V - U_{D3}}{I_{D3}} = \frac{5 - 1.8}{1 * 10^{-3}} = 3.2[kohm] \Rightarrow 3[kohm] \tag{1}$$

Maintenant, il faut dimensionner la résistance qui va sur ISet (2). Cette résistance sert à fixer le courant de charge. Pour cela, nous utiliserons la formule donnée dans la fiche technique : Ichg = Kiset/Riset.Nous devons d'abord déterminer le courant de charge Ichg, que l'on peut trouver dans la fiche technique d'une pile AAA NiMH. Ensuite, il faut trouver la valeur de Kiset, qui se trouve également dans la fiche technique.
Calcul :

$$R_{25} = \frac{K_{ISET}}{I_{CHG}} = \frac{300}{70 * 10^{-3}} = 4.286[kohm] \Rightarrow 4.3[kohm] \tag{2}$$

15. Fiche technique du BQ25172
16. Fiche technique du SI5618-TP

Ensuite, pour TMR (6), il s'agit de définir le temps de charge. Pour cela, il suffit de consulter le tableau fourni dans la fiche technique, qui indique les valeurs de résistance correspondant au temps de charge souhaité. Le temps de charge peut être trouvé dans la fiche technique d'une pile AAA NiMH .

Dans mon cas, j'ai besoin d'un temps de charge de 16 heures qui correspond à une résistance de pull-down de 8,2 kohm.

Pour finir, Vset (7) sert à indiquer au système le nombre de cellules utilisées. Pour cela, il suffit de consulter le tableau dans la fiche technique , qui donne les valeurs de résistance correspondant au nombre de cellules. Dans le système, j'utilise 4 cellules, ce qui correspond à une résistance de pull-down de 15 kohm.

Commande mosfet :
Pour le MOSFET, j'ai décidé de mettre arbitrairement une résistance de pull-down de 10 kohm pour garantir un niveau bas, et une résistance de 1 kohm sur la gate pour limiter le courant.

4.4.3 Schéma

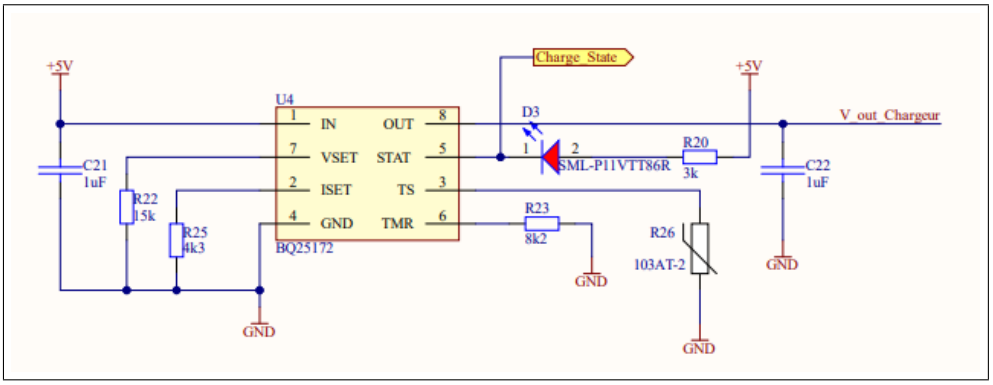


FIGURE 6 – Schéma du contrôleur de charge

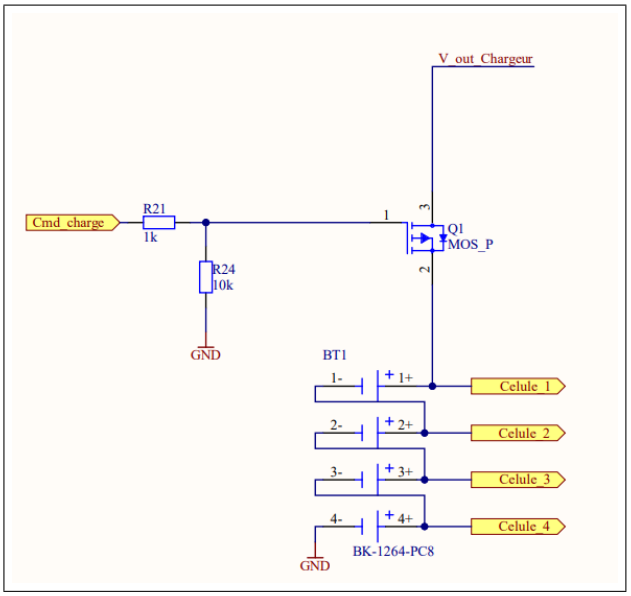


FIGURE 7 – Schéma de la commande mosfet

4.5 LED RGB

Pour les différentes indications une led RGB sera utilisée.

4.5.1 Choix principal

Pour ce qui est du modèle, j'ai repris le même utilisé dans l'ancienne version, le modèle APHF1608LSEEQBDZGKC. Pour le contrôle des LEDs, je vais les connecter aux sorties OC de mon microcontrôleur afin de pouvoir utiliser des PWM.

4.5.2 Dimensionnement

Tout d'abord, pour le dimensionnement, j'ai besoin de connaître la tension et le courant des LEDs rouge, bleue et verte. Ces informations se trouvent dans la fiche technique, figure 8 et 9. La LED rouge a une tension de 2,1V, tandis que les LEDs bleue et verte ont une tension de 3,1V. Le courant requis pour chacune des LEDs est de 20 mA.

Forward Voltage I_F = 2mA	V_F [2]	Hyper Red Blue Green	1.8 2.65 2.65	2.1 3.1 3.1	V

FIGURE 8 – Uout RGB

DC Forward Current	I_F [1]	30	20	20	mA

FIGURE 9 – Iout RGB

Calcul :
Pour dimensionner les résistances, j'utilise la loi d'Ohm.

$$R_{17} = \frac{3.3 - 2.1}{20 * 10^{-3}} = 600[ohm] \Rightarrow 560[ohm] \tag{3}$$

$$R_{18-19} = \frac{3.3 - 3.1}{20 * 10^{-3}} = 100[ohm] \tag{4}$$

4.5.3 Schéma

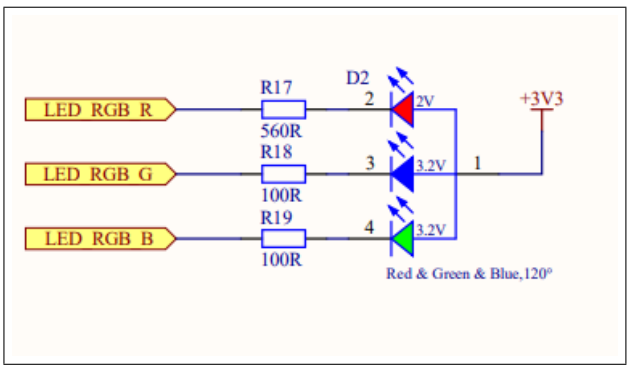


FIGURE 10 – Schéma de la led RGB

4.6 Capteur

Le capteur sera utilisé pour mesurer la température, l'humidité, la pression, ainsi que la qualité de l'air.

4.6.1 Choix principal

Le capteur utilisé sera le SEN-22858, une variante du modèle utilisé dans l'ancienne version. Le capteur sera connecté au microcontrôleur via I2C.

4.6.2 Dimensionnement

Pour le dimensionnement, étant donné qu'il s'agit d'un modèle sur PCB où tout est déjà présent sur la carte [17], j'ai simplement ajouté un condensateur de découplage de 100 nF.

4.6.3 Schéma

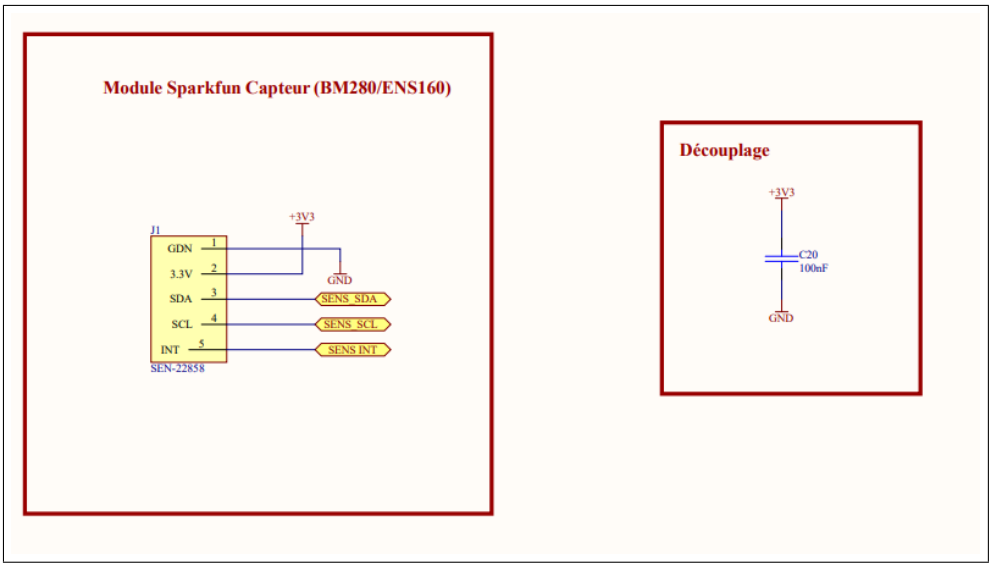


FIGURE 11 – Schéma du Capteur

17. Schéma du module SEN-22858

4.7 RTC

4.7.1 Choix principal

Pour la RTC, je suis resté sur le MCP74911 comment choisi lors de la pré-étude.

4.7.2 Dimensionnement

Pour le choix du quartz et des condensateurs, je me suis basé sur le schéma du kit de l'ETML-ES, figure 12.

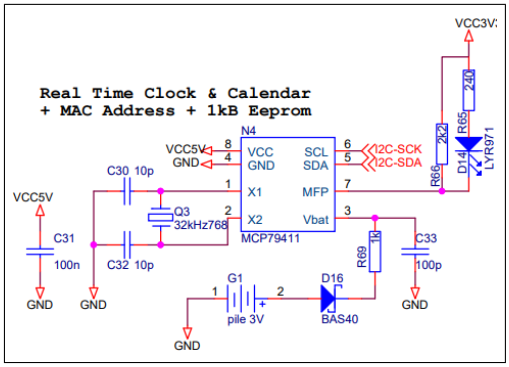


FIGURE 12 – Schéma de la RTC du kit de l'ETML-ES

Et des pull-up sans ajouter sur le SDA et le MFP, leurs valeurs sont de 10 k pour MFP et j'ai choisit aussi 10 k pour le SDA pour une fréquence de 100 kHz (l'I2C sera en low). Elles sont données dans le datasheet en page 8 du MCP74911, figure 13 et 14.

3.5 Multifunction Pin (MFP)

This is an output pin used for the alarm and square wave output functions. It can also serve as a general purpose output pin by controlling the OUT bit in the CONTROL register.

The MFP is an open-drain output and requires a pull-up resistor to VCC (typically 10 kΩ). This pin may be left floating if not used.

FIGURE 13 – Info pin MFP

3.3 Serial Data (SDA)

This is a bidirectional pin used to transfer addresses and data into and out of the device. It is an open-drain terminal. Therefore, the SDA bus requires a pull-up resistor to VCC (typically 10 kΩ for 100 kHz, 2 kΩ for 400 kHz). For normal data transfer, SDA is allowed to change only during SCL low. Changes during SCL high are reserved for indicating the Start and Stop conditions.

FIGURE 14 – Info pin SDA

4.7.3 Schéma

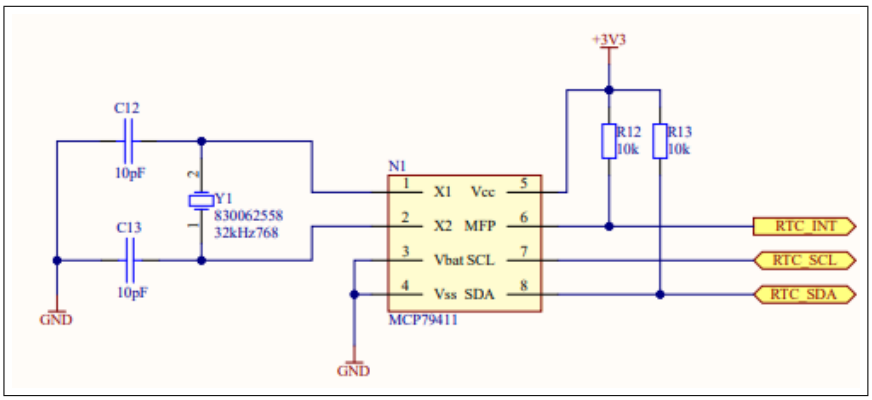


FIGURE 15 – Schéma de la RTC

4.8 Module Wifi - ESP32

4.8.1 Choix principal

Un module Wi-Fi l'envoie de données sans fil sur un serveur MQTT. comme choisi durant la pré-étude. Le module « ESP32-C3 [18] » a été sélectionné au vu des projets réalisés avec lui au sein de l'ES. Cela me permet de bénéficier de la documentation des anciens étudiants ou des enseignants.

4.8.2 Dimensionnement

Pour les liaisons, je me suis basé sur le schéma typique du module qui se trouve dans le Datasheet, figure 11 16.

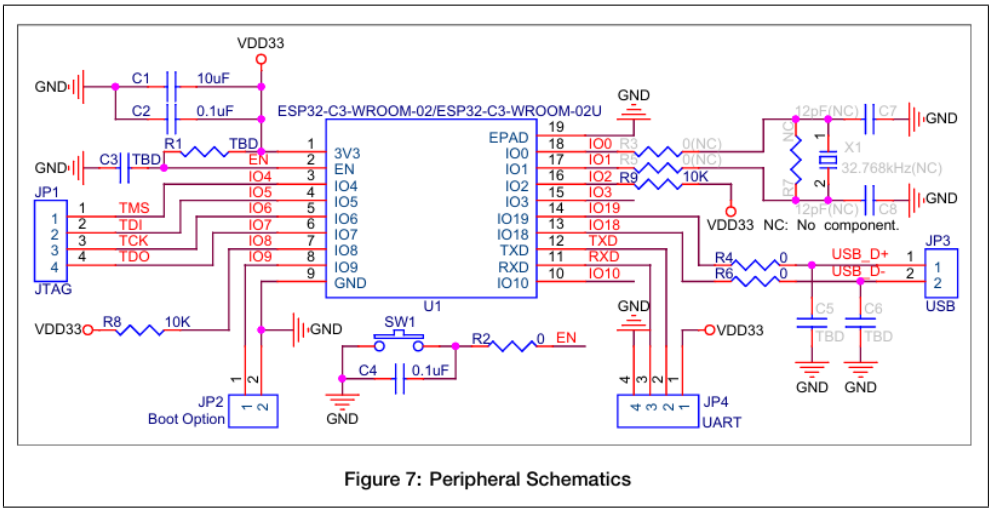


Figure 7: Peripheral Schematics

FIGURE 16 – Schéma typique

Pour la plupart des recherches, elles étaient faites sur le site internet du fabricant ESPRESSIF qui met à disposition un user guide [19].

Grâce au tableau dans la figure 17 qui se trouve dans l'User guide section Get Started/Hardware Connection, je peux voir les connexions en lien avec l'UART. On ressort de ce tableau que l'UART0 se trouve sur les GPIO20 (RX) et GPIO21(TX), l'UART0 sert à la configuration du module qui se fait pour pouvoir l'utiliser. Et pour la communication avec le microcontrôleur, on utilise l'UART1 qui se trouve sur les GPIO6 (RX) et GPIO7 (TX).

ESP32-C3 Series Hardware Connection Pinout

Function of Connection	ESP32-C3 Board or Module Pins	Other Device Pins
Download/Log output [1]	UART0 • GPIO20 (RX) • GPIO21 (TX)	PC • TX • RX
AT command/response [2]	UART1 • GPIO6 (RX) • GPIO7 (TX) • GPIO5 (CTS) • GPIO4 (RTS)	USB to serial converter • TX • RX • RTS • CTS

FIGURE 17 – Pinout UART

18. Fiche technique de l'ESP32
19. User guide ESP32

Pour la configuration externe de l'esp, des GPIO précis sont utilisés pour mettre l'esp32 en mode download, pour cela on trouve ces informations grâce au datashet, dans lequel on peut trouver un tableau avec les GPIO et l'état de la pin, figure 18. Les GPIO2 et 8 seront contecter à des pull-up de 10 k, même valeur que le schéma typique (figure 16) et un switch sera mis le GPIO9 pour mettre la pin à l'état bas pour la configuration de l'ESP32.

Booting Mode [1]			
Pin	Default	SPI Boot	Download Boot
GPIO2	N/A	1	1
GPIO8	N/A	Don't care	1
GPIO9	Internal weak pull-up	1	0

FIGURE 18 – Boot mode GPIO

Et pour finir, un bonton reset sera mis : il sera inspiré par le schéma typique (figure 16) et pour les valeurs de la resistance du condo, il est donné par le datasheet, figure 19.

- To ensure that the power supply to the ESP32-C3 chip is stable during power-up, it is advised to add an RC delay circuit at the EN pin. The recommended setting for the RC delay circuit is usually R = 10 kΩ and C = 1 μF. However, specific parameters should be adjusted based on the power-up timing of the module and the power-up and reset sequence timing of the chip. For ESP32-C3's power-up and reset sequence timing diagram, please refer to *ESP32-C3 Series Datasheet* > Section *Power Scheme*.

FIGURE 19 – Reset RC

Maintenant que nous savons sur quel GPIO se trouvent les UART, nous pouvons trouver la correspondance sur les pins avec le tableau des pin definitions en page 10-11 dans le datasheet de l'ESP32 (annexe ?). Avec le tableau 4 voici les pins pour les GPIO que je vais utiliser.

Pin	**GPIO**	**Fonction**
GPIO2	16	Boot mode
GPIO6	5	UART1 RX
GPIO7	6	UART1 TX
GPIO8	7	Boot mode
GPIO9	8	Boot mode
GPIO20	11	UART0 RX
GPIO21	12	UART0 TX

TABLE 4 – Table des pin

4.8.3 Schéma

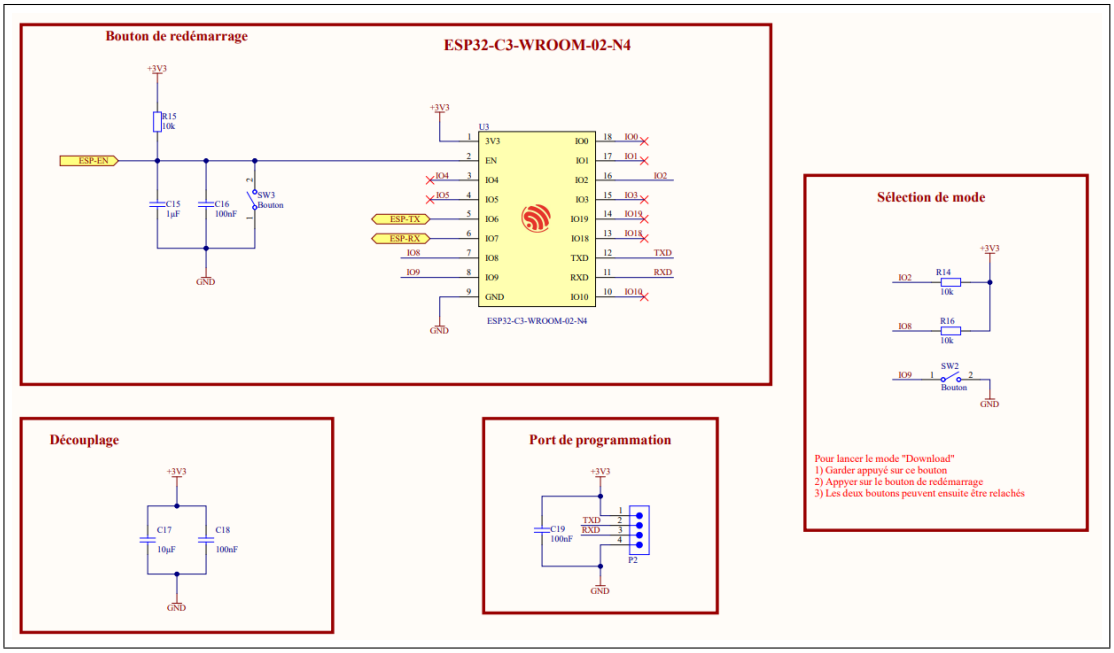


FIGURE 20 – Schéma de l'ESP32

4.9 Microcontrôleur

4.9.1 Choix principal

Je suis parti sur le PIC32MX130F256D, qui est le même modèle que dans la version précédente, mais avec plus de broches, parce que la version en-dessous ne possède pas assez de pins pour tout les connecter.

4.9.2 Dimensionnement

Aucun calcul n'a été requis, car la documentation du microcontrôleur fournit le schéma minimal nécessaire pour le faire tourner. Par rapport au schéma du datasheet un switch a était rajouté pour le reset sur la pin MCLR.

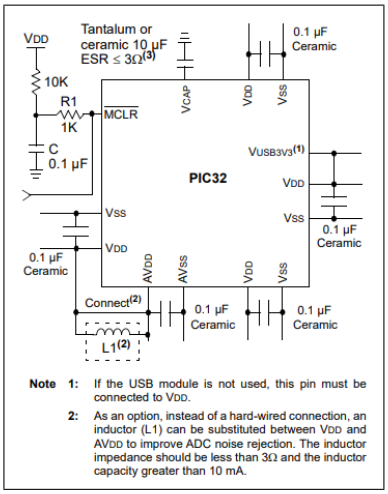


FIGURE 21 – Schéma tipique du uC

4.9.3 Schéma

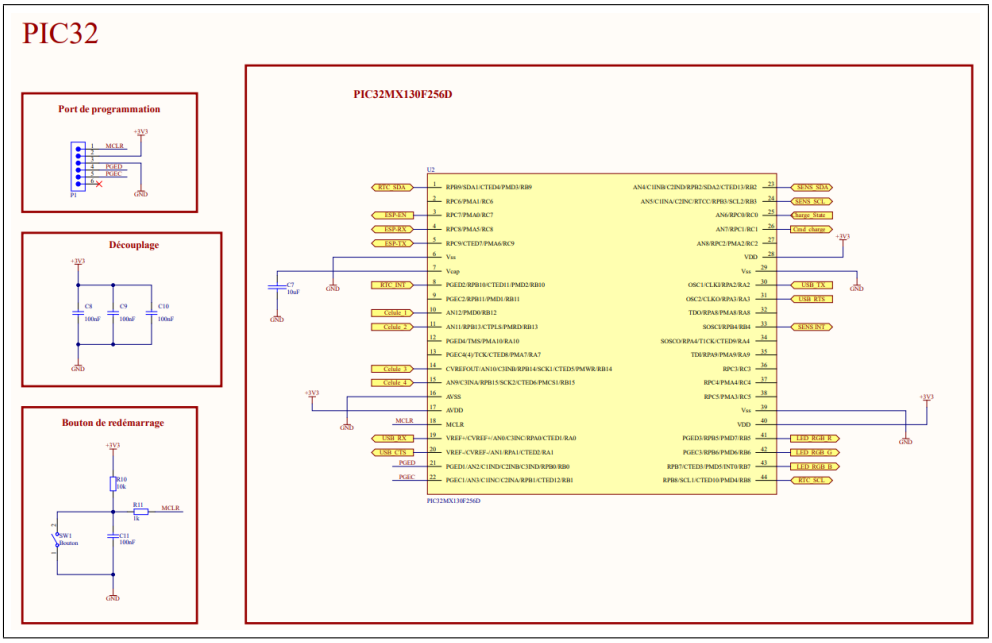


FIGURE 22 – Schéma du uC

5 Design circuit imprimé

5.1 Introduction

Une fois le schéma terminé, j'ai débuté la phase de design du circuit imprimé sur Altium. J'ai dû respecter différentes contraintes telles que la taille du PCB, des pistes et de l'emplacement du module Wifi aisni que le support des piles AAA Les détails sont décrits dans les sections suivantes.

5.2 Caractéristiques du circuit

Aillant une contrainte au niveau des dimensions de la carte, elle est de 60 mm x 60 mm. De ce fait, j'ai peu d'espace et il faudra faire attention au niveau du placement des composants. Malgré le peu de place, je peux sans problème travailler uniquement sur deux couches (top et bottom) de 35 µm d'épaisseur de cuivre chacune. Le circuit est produit en matériau FR4 qui est le plus commun pour les circuits de base (circuits n'ayant pas de signaux hautes fréquences ou autres spécialités). La quasi totalité des composants est placée sur le côté TOP du PCB. Le support de piles sera le seul composant au bottom, à cause de cela les composants THT devront être placés sur les bord du PCB pour éviter le support au bottom.

La largeur des pistes a été en premier lieu trouvée grâce au tableau graphique d'Eurocircuit, figure 23 . Pour les pistes, les normales seront de 0.25 mm et pour l'alimentation, j'ai ajouté une petite marge qui fera une largeur de 0.35 mm. J'ai également décidé d'augmenter la largeur pour les deux lignes différentielles de l'USB-C à 0.38 mm.

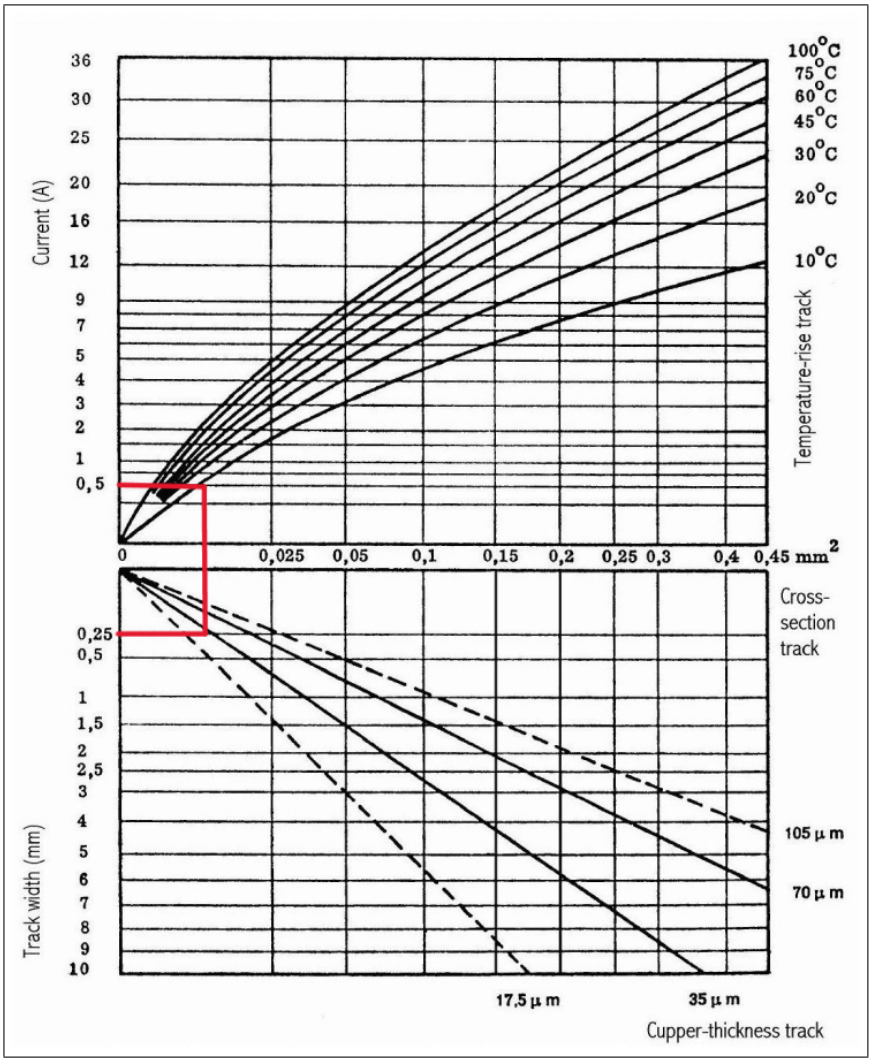


FIGURE 23 – Graphique d'Eurocircuit

5.3 Placement et routage des composants

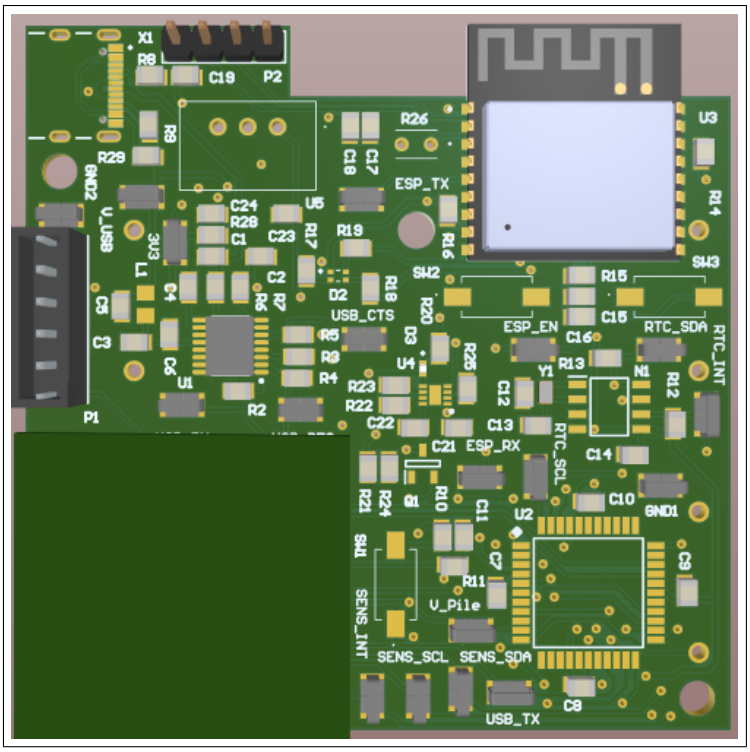


FIGURE 24 – 3D du PCB

Sur la figure 24 on voit le rendu 3D du PCB. j'ai tout d'abord placé le support de pile sur le coté bottom pour ensuite pouvoir placer les composant avec des patte traversant. avant le placement des des composant THT il faut faire intention avec le placement du module wifi ESP32, des diretice sont donné dans l'user guide, figure 25 et 26.

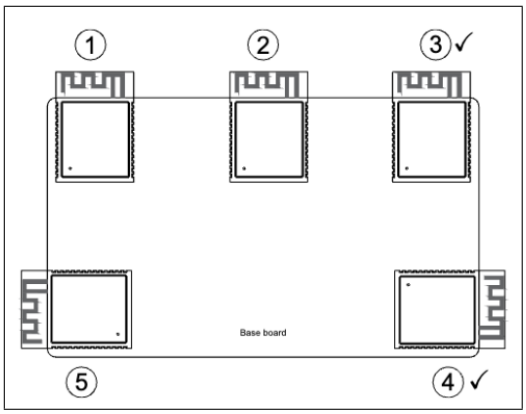


FIGURE 25 – Placement zone

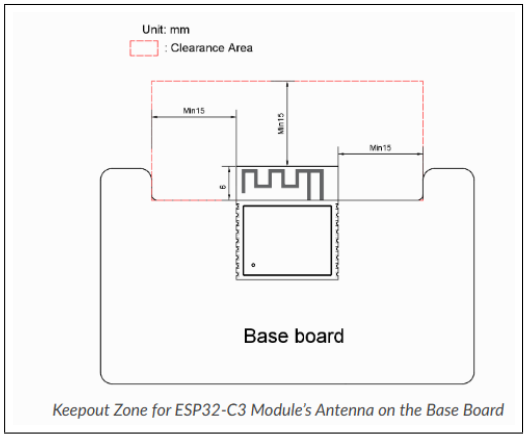


FIGURE 26 – Keepout zone

On respectons les directives de l'ESP. J'ai placé le composant en haut à droite et coupé une partie du PCB pour que l'antenne de l'ESP32 soit dans la keepout zone (min 15 mm). Suite à ça, j'ai placé le connecteur USB-C et le régulateur en haut à gauche, le connecteur pour l'ESP juste à côté de l'USB-C, le connecteur de programmation sur le côté gauche et également le module avec les deux capteurs de Sparkfun tout en faisant attention que leurs pattes ne rentraient pas en conflit avec le support de piles.

Ensuite, pour la thermitance (R26), je le place juste en face du trou qui donne sur les piles et à la limite du support de pile. Le but est de rabattre le composant dans le trou pour qu'il soit au plus proche des piles.

Pour finir, j'ai fait des petits blocs avec les composants restants en suivant la schématique et en faisant intention au condosateur de découplage pour qu'il soit au plus des alimentations des composants. Un trou a été ajouté entre le connecteur USB-C et le connecteur de programmation pour pouvoir venir fixer le PCB dans le boitier. Un second trou est placé en bas à droite où un support viendra s'appuyer pour maintenir le PCB.

Sur les figures 27 et 28 on peut voir le top et le bottom du PCB.

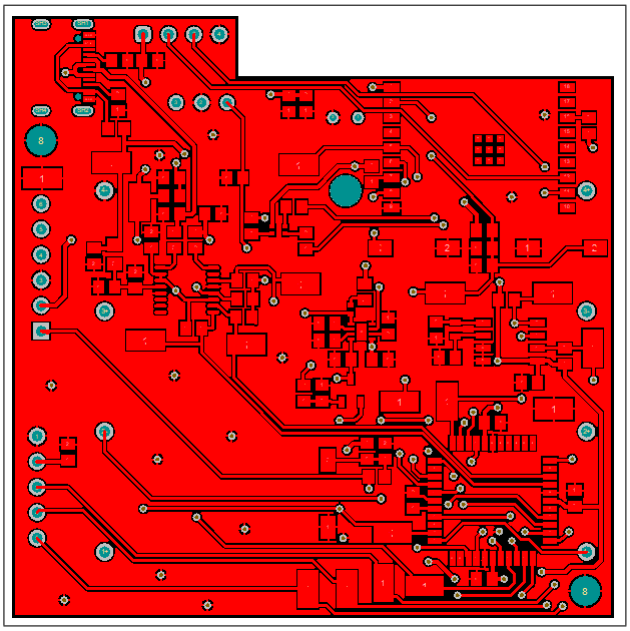


Figure 27 – Vue top du PCB

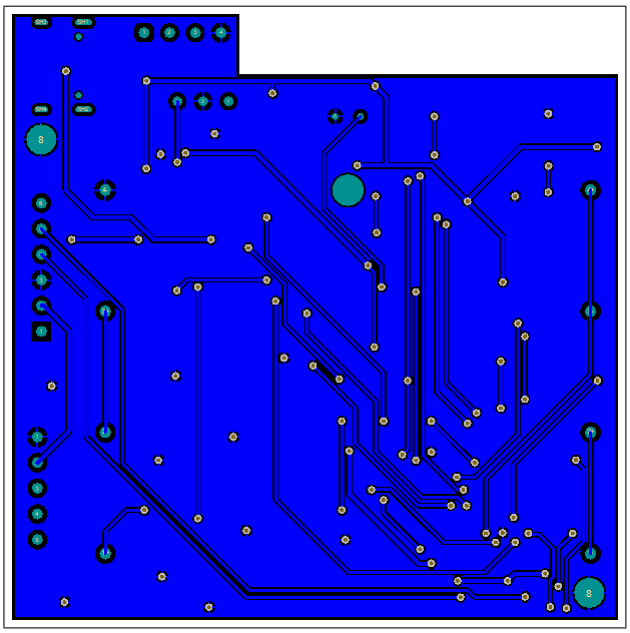


Figure 28 – Vue bottom du PCB

6 Programmation de l'ESP32

6.1 Introduction

Pour pouvoir utiliser les commande AT de l'ESP32 plus tard avec le microcontroleur il faut tout d'abord le configuré grace au connecteur externe et un cable USB-UART de chez FTDI (TTL-232R-3V3 [20]).

6.2 Prérequis

Télécharger le programme ESP-AT sur la page internet User guide [21], figure 29 .

FIGURE 29 – Firmware AT de l'ESP32

Prendre le fichier du dossier du projet ou télécharger l'outil de programmation sur le lien suivant : Télécharger l'outil de programmation sur le site de ESPRESSIF [22], figure 30 .

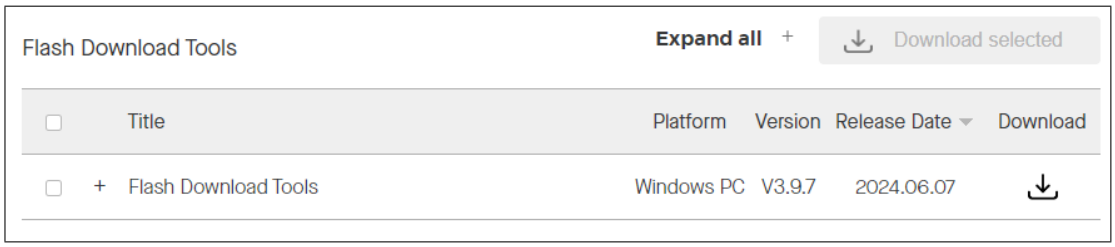


FIGURE 30 – Outil de programmation

6.3 Schéma

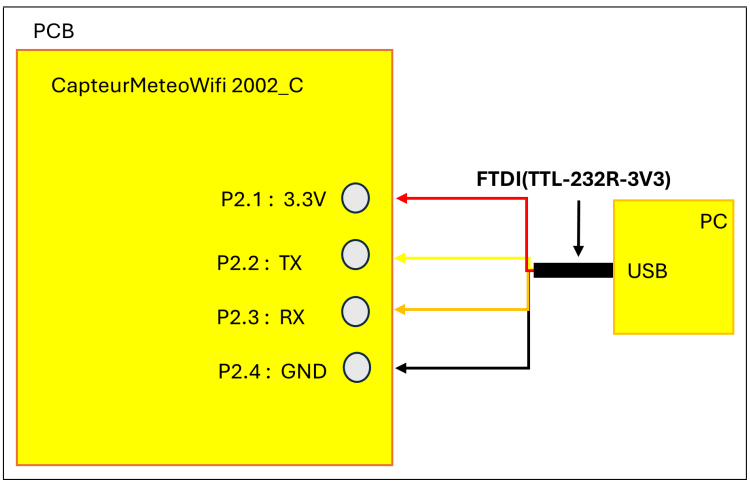


FIGURE 31 – Schéma pour la configuration de l'ESP32

20. fiche technique du TTL-232R-3V3
21. Lien pour le firmware AT ESP32
22. Lien pour le l'outil de flash

6.4 Marche à suivre

6.4.1 Mettre l'esp en mode download

— Brancher la carte selon le schéma ci-dessus .
— Dans le PC, ouvrir le gestionnaire de périphériques.
— Connecter l'interface au port USB du PC et récupérer son nom de port.
— Dans le PC, ouvrir PuTTY.
— Sur PuTTY, sélectionner la communication série et configurer les options. selon la figure32.

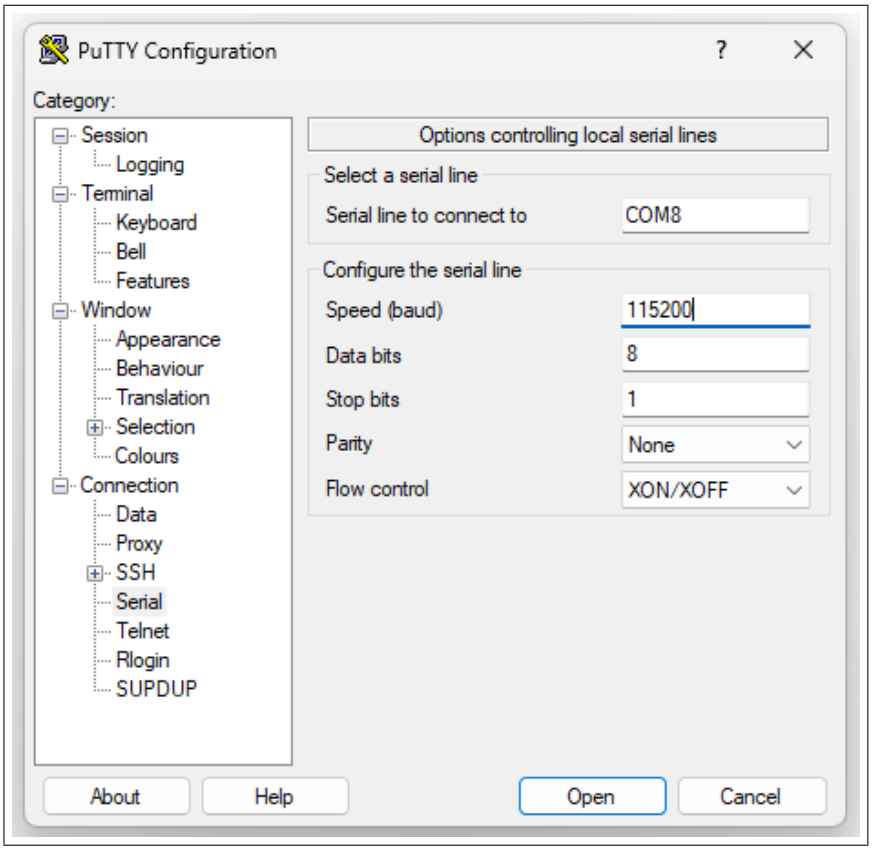


FIGURE 32 – Putty configuration

— Cliquer sur le bouton Open.
— Maintenir appuyer sur le switch SW2.
— Apuyer sur le switch SW3.
— Sur PuTTY le message sur la figure 33 s'affiche.

FIGURE 33 – Putty message

— Fermer PuTTY

6.4.2 Programmation de l'ESP32

— Ouvrir l'outil de programmation téléchargé dans le prérequis.

— Un fois ouvert sélectionner la configuration sur la figure 34.

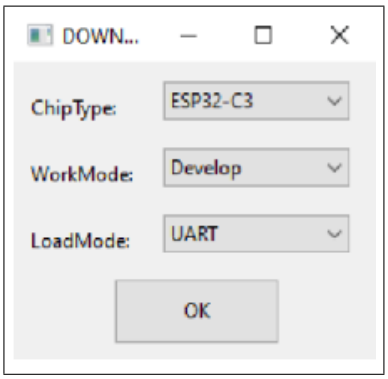


FIGURE 34 – Configuration logiciel de flash

— Appuyer sur OK.

— Un fois ouvert sélectionner la configuration sur la figure 35.

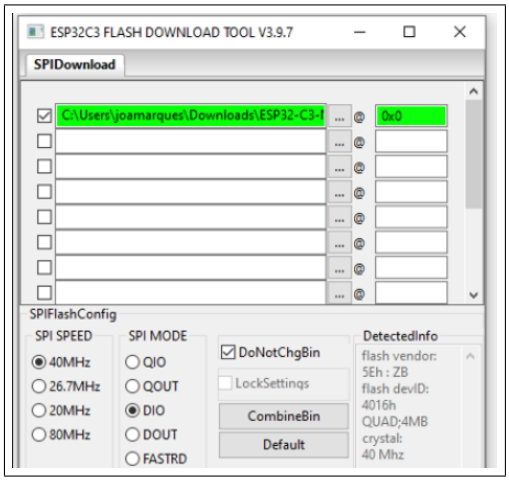


FIGURE 35 – Configuration logiciel de flash

— Cochez la case de la première ligne, sélectionnez le fichier « factoryMINI-1.bin », fichier telecharger dans le prérequis.

— Puis mettre la valeur 0x0 dans la colonne de droite

— Cochez « DoNotChgBin »

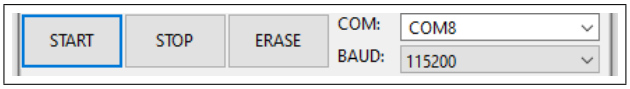


FIGURE 36 – Configuration logiciel de flash

— Comme sur la figure 36 sélectionner le port COM utilisé sur PuTTY et réglez le baudrate a 115'200 Bd

— Cliquez sur le bouton « Start » pour démarrer la programmation, la programmation prend quelques minutes.

A la fin de la programmation on aura l'affichage suivant sur la figure 37.

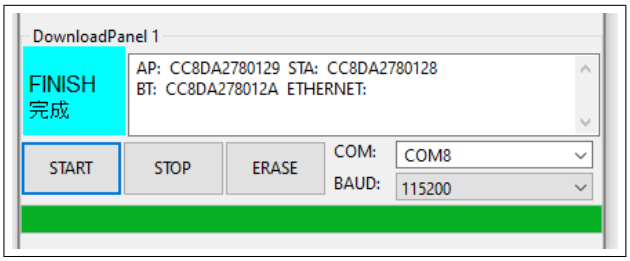


FIGURE 37 – Installation fini

6.4.3 Vérification de la programmation

- Dans le PC, ouvrir PuTTY et régler de la même façon que le point 4.3.1 avec Flow Control = None
- Ouvrir le port.
- Apuyer sur le switch SW3.
- Si l'ESP32 a été bien programmée, le message suivant doit s'afficher comme sur la figure 38.

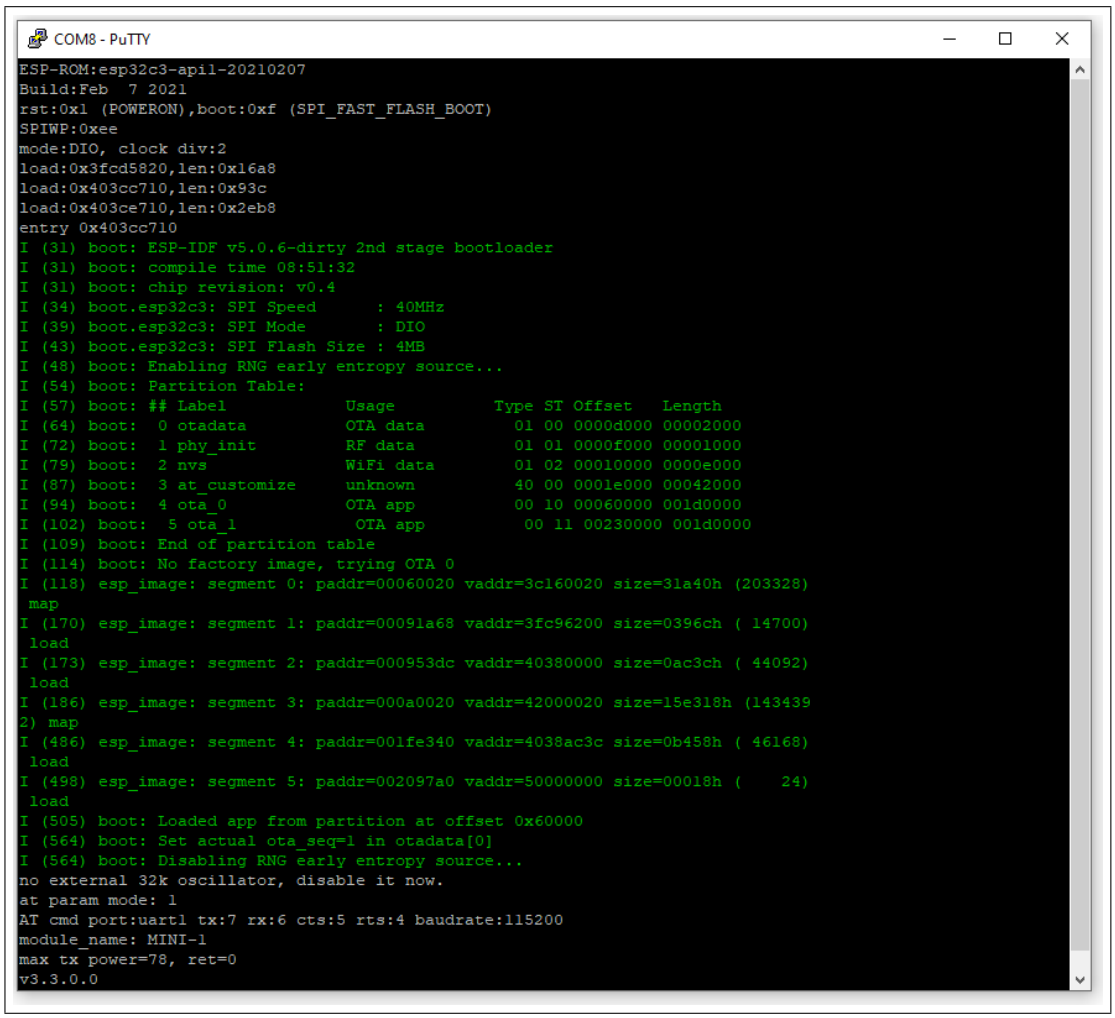


FIGURE 38 – Vérification

7 Montage et tests

7.1 Test BMe280, ENS160, MCP74911

En attendant de recevoir mes composants ainsi que mon PCB, j'ai pu tester mes capteurs BME280 et ENS160 (module connectable par câble) ainsi que la RTC MCP74911 (même modèle sur le kit PIC32) sur le kit PIC32 de l'ETML.

Pour BME280, j'ai tout simplement repris le code de l'ancienne version et il est fonctionnel, donc pour la suite du projet je pense que je pourrais récupérer les fichiers BME280.c et BME280.h et j'y ferais quelques modifications.

Ensuite, pour le ENS160, j'ai pu communiquer avec lui et récupérer les valeurs de qualité d'air. Pour les tests, je n'ai pas réalisé de fonction, mais juste utilisé les fonctions de communication I2C. Plus tard, sur le vrai PCB, des fonctions seront réalisées pour l'initiation, la lecture et la récupération des valeurs.

Pour finir, avec le MCP74911, j'ai pu démarrer l'horloge de la RTC, j'ai pu commencer a tester à activer l'alarme mais toujours pas fonctionnel.

7.2 Montage

La carte a été monter sur 2 jours, tout d'abord j'ai commencer par monter les composant compliquer a monter tel que BQ25172 qui est un boîtier type WON-8, ensuite je suis parti sur le module ESP32, le connecteur USB-C.

Par la suite j'ai monter mon alimentation et contrôlé que j'avais bien mes 3.3V pour le reste du montage, pour finir j'ai monter les composant SMD restant et ensuite les composant THT.

7.3 Amélioration

Avant et après montage de mon PCB j'ai put constater quelque erreur sur mon PCB certain on pu être résolu et d'autres en était laisser de coté pour l'instant et recherche seront durant la préparation de la présentation.

Tout d'abord, voici les différentes erreurs trouvées avant montage :

Pour la RTC, j'ai fait une erreur au niveau de la schématique où j'ai inversé les pin 5 à 8. Cela était corriger sur le PCB : on ne brasait que la partie gauche du composant et en surlevant la partie droite, et ensuite, je suis venu connecter les pin 5 à 8 avec des petits fils. La modification sur Altium sera faite après le rendu du rapport de diplôme.

Ensuite, pour la lecture ADC sur les cellules, je me suis rendu compte que l'ordre de préparation de la configuration des pins de mon microcontrôleur avait oublié un point important. Pour les cellules 3 et 4, il n'y a pas de problème, mais pour les cellules 1 et 2, le problème vient de la tension de sortie qui sera supérieure à 3,3 V, donc supérieure à la tension maximale de l'ADC. Pour corriger cela, il suffit de mettre des ponts diviseurs pour diminuer cette tension sous les 3.3 V.

Pour le PCB présent, j'ai simplement coupé les pistes sur la cellule 1 et 2, et pour la modification Altium, tout comme la RTC, elle sera faite après.

Et pour finir, l'erreur trouvée après montage et qui sera laissée pour plus tard :

J'ai pu remarquer après quelques problèmes avec mon chargeur de pile NiHM, tout d'abord Vout quand le câble USB-C est branché et qu'il n'y a pas de pile, elle n'est pas de 5 V. Premier problème le circuit pour fonctionner avoir des pile de brancher ensuite dans le cas ou le câble et brancher et que des piles NiMH aussi sans monter le module mes la led d'état a clignoter qui veut dire que le système a un problème, au lieu d'être sois allumer ou éteinte.

La première hypothèse serait que le composant soit mal monté vu que son boitier est de type WON-8, ce qui pourrait être le cas mais la question doit être étudiée plus en profondeur après rendu du rapport.

8 Firmware

8.1 Introduction

La partie firmware décrit la programmation qui a été réalisée pour ce produit. Elle répertorie les principales informations sur les différentes librairies utilisée.

Un listing complet du code réalisé est disponible en annexe ?.

8.2 Configuration Harmony

Pour cette partie, j'ai commencé par créer un projet "Microchip Embedded" "32-bit MPLAB Harmony Project" compatible avec le PIC32MX130F256D

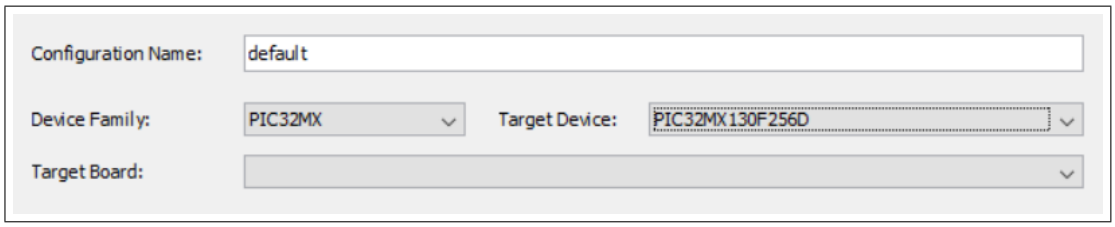


Figure 39 – Harmony création de projet

Directement dans Harmony Configuration, j'ai commencé par modifier le DEVCFG0 afin d'activer le mode Debug.

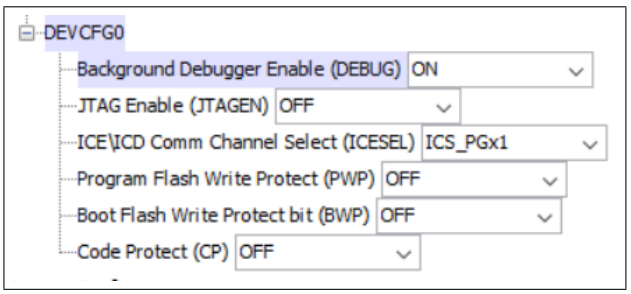


Figure 40 – DEVCFG0

Ensuite je suis ensuite allé dans le Pin Settings afin d'assigner le nom des pins ainsi que leurs fonctions. Avec le configurateur graphique on peut mieux voir le pin out fais sur mon microcontroleur, figure 41

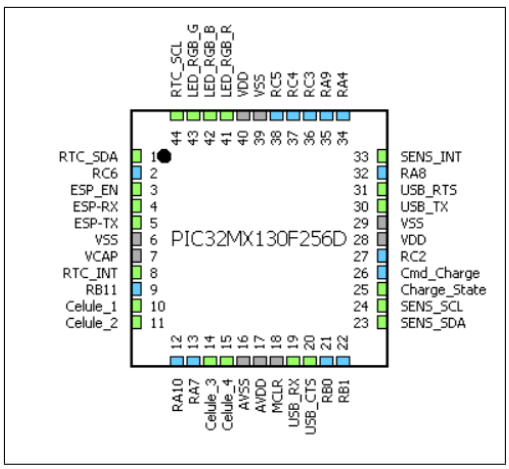


Figure 41 – Pinout du PIC32MX130F256D

8.2.1 Configuration des périphériques

Pour la communication en I2C et les ADC, nous n'avons pas besoin de les configurer grâce aux drivers de Harmony et en va utiliser les librairie a disposition que nous avons déjà utiliser lors des cours minf sur le kit PIC32 de l'École.

Pour les UARTs, voici les réglages effectués.

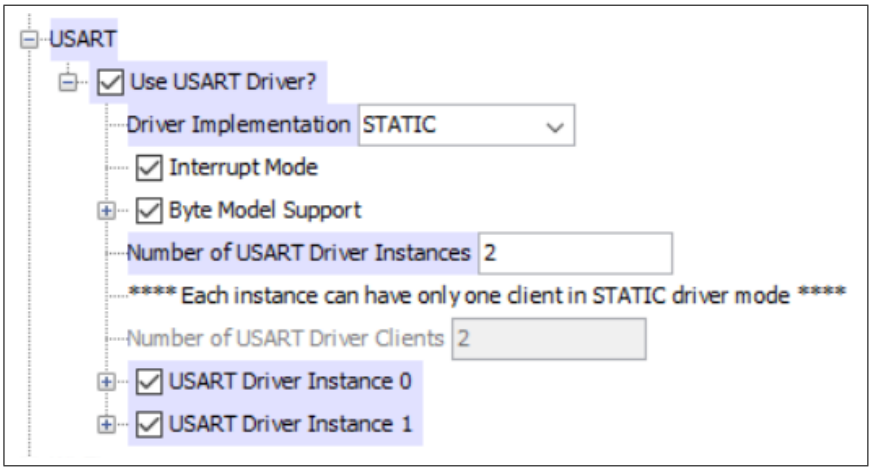


FIGURE 42 – Configuration UART

La première instance représente l'UART utilisé pour la communication en le microcontrôleur et le module wifi. Et la deuxième instance représente la communication entre le microcontrôleur et l'ordinateur.

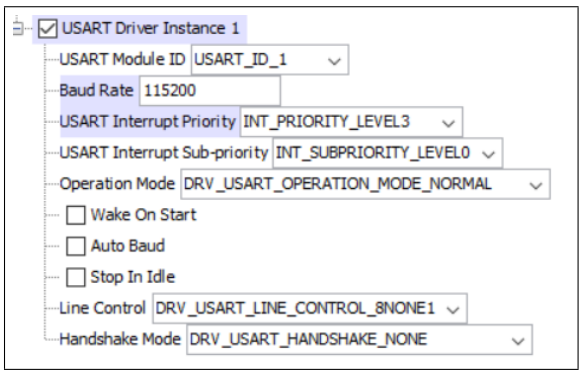


FIGURE 43 – Configuration UART1

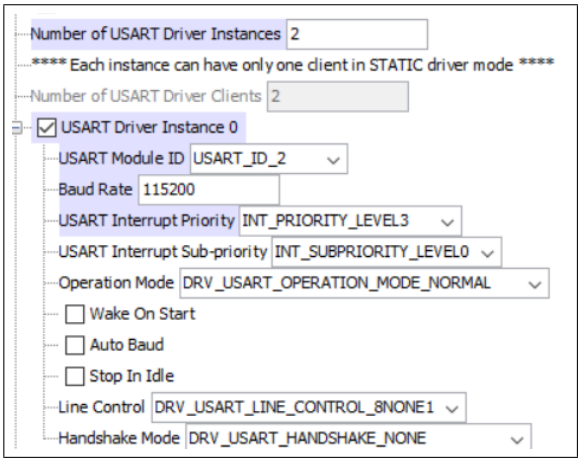


FIGURE 44 – Configuration UART2

Ensuite j'ai activé le Timer et son interruption (toute les 50ms). La valeur de recharge ainsi que le prescaler ont étés calculé de la manière suivante :

$$Prescaler = \frac{TempTimer * Frquence}{NbBitTimer} = \frac{50 * 10^{-3} * 20 * 10^{6}}{2^{16}} = 15.26 -> 16 \tag{5}$$

$$TimerPeriod = \frac{TempTimer * Frquence}{prescaler} - 1 = \frac{50 * 10^{-3} * 20 * 10^{6}}{16} - 1 = 62499 \tag{6}$$

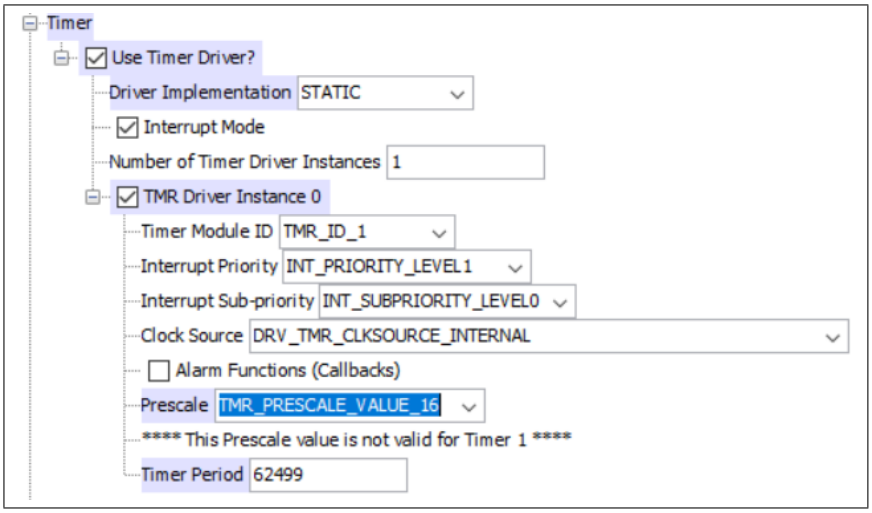


FIGURE 45 – Configuration Timer

8.3 Librairies utilisées

Nom	Description
app	Gestion de la machine d'état global
GestMenu	Gestion de la machine d'état du menu et c'est fonction
BME280	Fonctions pour gérer le capteur BME280
ENS160	Fonctions pour gérer le capteur ENS160
ESP32	Fonctions pour gérer le module Wifi ESP32
MCP74911	Fonctions pour gérer la RTC MCP74911
Mc32NVMUtil	Fonctions pour l'écriture et lecture de la mémoire flash de l'ETML-ES
Mc32DriverAdc	Fonctions pour la lecture ADC de l'ETML-ES
Mc32Delays	Fonctions de délais de l'ETML-ES
Mc32gestI2c	Gestion de communication I2C de l'ETML-ES
Mc32gestRS232	Gestion de la communication UART de l'ETML-ES
GesFifoTh32	Gestion de la fifo de l'ETML-ES
Mc32I2cUtilCCS	Utilitaire I2C de l'ETML-ES

TABLE 5 – Liste de librairie

Dans le tableau 5 on peut voir les librairie utilisée dans la conception. Une grande partie des librairie sont des librairie récupérer de l'ETML-ES et de l'ancienne version. Dans les section suivante je parlerais des librairie que j'ai crée ou modifier.

8.4 BME280

Pour le capteur BME280 je vais utiliser les fichier BME280.c/BME280.h, il dispose des fonction listé dans le tableaux 6.

Fonction	**Description**	**Ligne de début-fin**
BME280Init	Initialisation du BME280	78-134
BME280ReadCoefficents	Lecture des coefficients de calibration	147-171
BME280readTemperature	Lecture de la température	185-207
BME280readPressure	Lecture de la pression	221- 254
BME280readHumidity	Lecture de l'humidité	268-296
BME280HumidityStatus	Détermine le status d'humidité	308-318
BME280Forecast	Détermine la prévision météo	331-419

Table 6 – Liste des fonction de la librairie BME280

Cette libraire a été crée par l'élève Rickenbach pour l'ancienne version du produit. je vais utiliser les fonction tel que tel vu quel sont fonctionnel.

8.5 ENS160

Pour le capteur ENS160 je vais utiliser les fichier ENS160.c/ENS160.h, il dispose des fonction listé dans le tableaux 7.

Fonction	**Description**	**Ligne de début-fin**
SwapMSBLSB	Inverse les MSB et les LSB	34-47
ENS160Init	Initialisation du ENS160	59-79
ENS160ReadDataAqi	Lecture de l'indice de qualité d'aire	89-98
ENS160ReadDataTvoc	Lecture de la concentration de particule	108-121
ENS160ReadDataEco2	Lecture de al concentration de CO2	131-144

Table 7 – Liste des fonction de la librairie ENS160

Cette librairie a été crée par moi est a été inspirée de la librairie du BME280. Les informations telles que les adresses de registre ont été trouvées dans la datasheet du ENS160 [23].

23. Fiche technique du BME280

8.5.1 ENS160Init

Cette fonction commence par la lecture de l'ID est ensuite configure le capteur ENS160 pour le mettre en mode de fonctionnement standard et appliquer les paramètres de configuration. Elle inclut également un délai pour permettre au capteur de se stabiliser après la configuration.

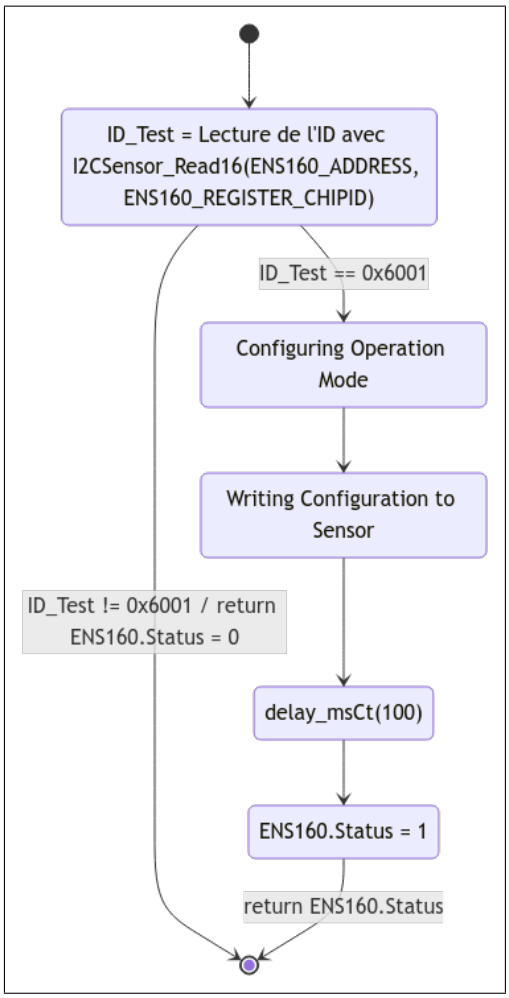


FIGURE 46 – Fonction Ens160Init

8.5.2 ENS160ReadDataAqi

Cette fonction vient lire l'indice de qualité de l'air et retourne la valeurs en paramètre de sortie.

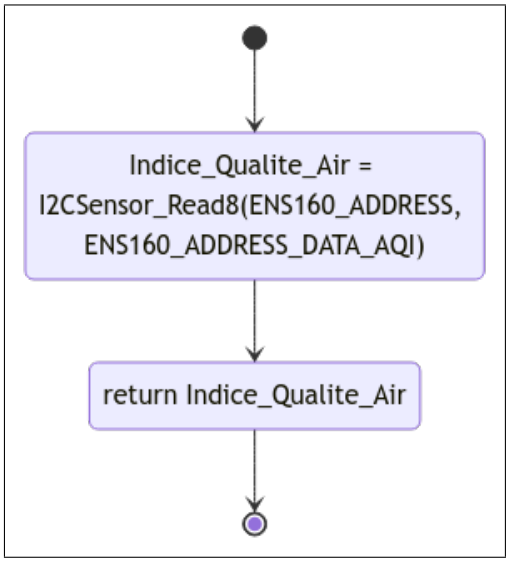


FIGURE 47 – Fonction Ens160ReadDataAqui

8.5.3 ENS160ReadDataTvoc et ENS160ReadDataEco2

Les deux fonctions viennent lire les concentrations de TVOC pour l'une et de CO2 pour l'autre. Les données lues sont ensuite ajustées en inversant les MSB et LSB et les retournent comme paramètre de sortie.

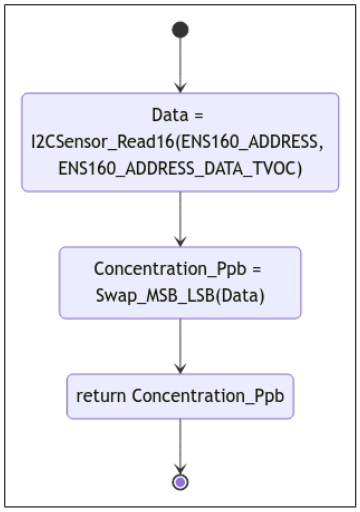


FIGURE 48 – Fonction ENS160ReadDataTvoc

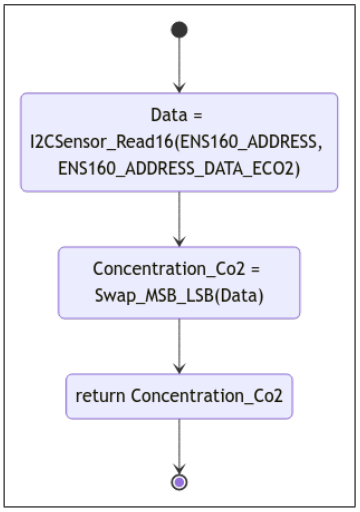


FIGURE 49 – Fonction ENS160ReadDataEco2

8.5.4 SwapMsbLsb

Cette fonction inverse la valeur en paramètre d'entrée, les octets MSB et LSB de la valeur en 16 bit en paramètre d'entrée, et ensuite la retourne en paramètre de sortie.

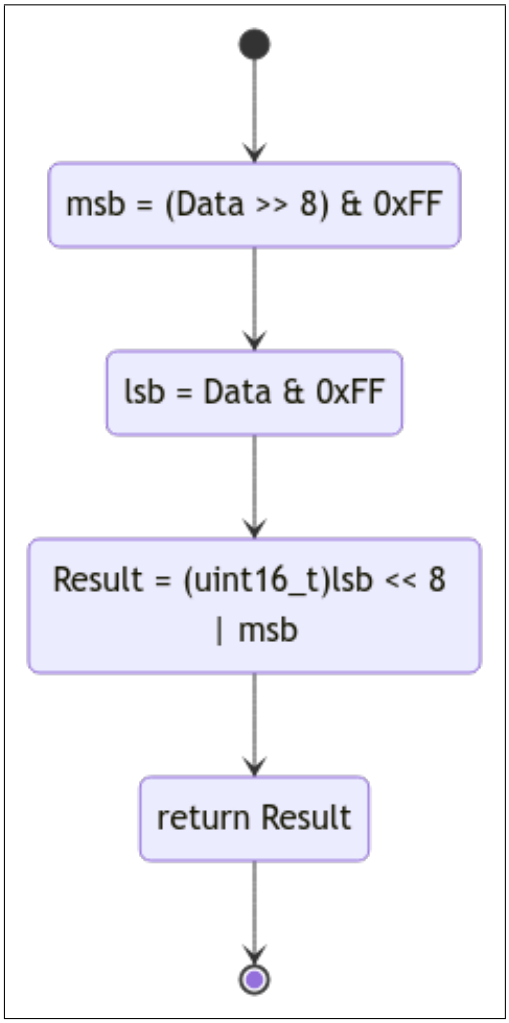


FIGURE 50 – Fonction Ens160SwapMsbLsb

8.6 ESP32

Pour le module wifi EPS32 je vais utiliser les fichier ESP32.c/ESP32.h, il dispose des fonction listé dans le tableaux 8.

Fonction	Description	Ligne de début-fin
ESP32Init	Initialisation de l'ESP32	52-99
SendATCommand	Envoie des command AT	112-118
ReadIpAndMac	Lecture de l'IP et l'adresse MAC	129-151
ESP32isStarted	Envoie une commande AT de vérification	165-179
ESP32ConnectWifi	Connexion au Wi-fi	198-216
ESP32ChangeMode	Change le mode l'ESP32	235-251
ESP32GetIpAndMac	Récupération de l'IP et de l'adresse MAC	270-285
PublishMQTT	Envoie les data sur le serveur MQTT	300-308
SetupMQTTAndPublish	S'occupe des paramétrages et de l'envoie	323-352
ESP32ConnectToTCPServer	Connexions au serveur TCP	366-392
ESP32DisconnectTCPServer	Déconnexion au serveur TCP	404-414
ESP32SendTCP	Envoie de message au serveur TCP	427-443

Table 8 – Liste des fonction de la librairie ESP32

Cette librairie a été crée par moi est a été inspirée de la librairie de l'ESP8266 crée par l'élève Rickenbach pour l'ancienne version du produit.

8.6.1 SendATCommand

Cette fonction envoie une commande AT au module Wi-Fi graec a la fonction SendWifiMessage, et attend un délai pour s'assurer que la requête est correctement envoyée et que la réponse est reçue.

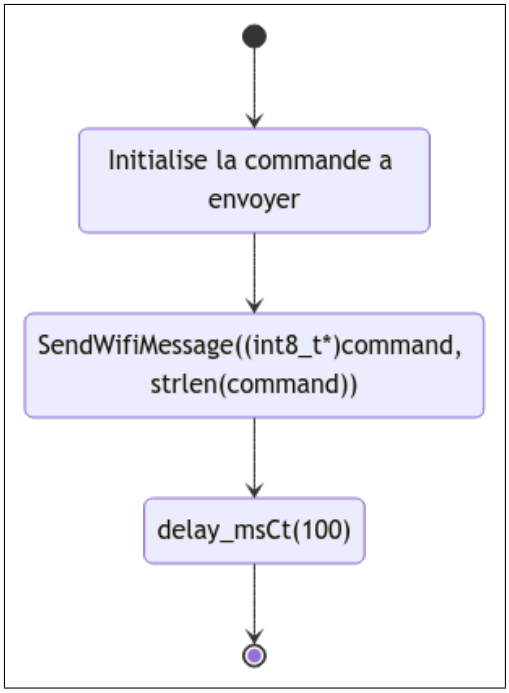


Figure 51 – Fonction SendATCommand

8.6.2 ESP32Init

Cette fonction initialise le module ESP32 en configurant les paramètres Wi-Fi, en activant l'ESP32 et effectue les vérifications nécessaires. Elle tente de se connecter au réseau Wi-Fi spécifié et change le mode de l'ESP32 en mode Station. La fonction récupère également l'adresse IP et l'adresse MAC de l'ESP32 et met à jour le statut en fonction de la validité de l'adresse IP obtenue.

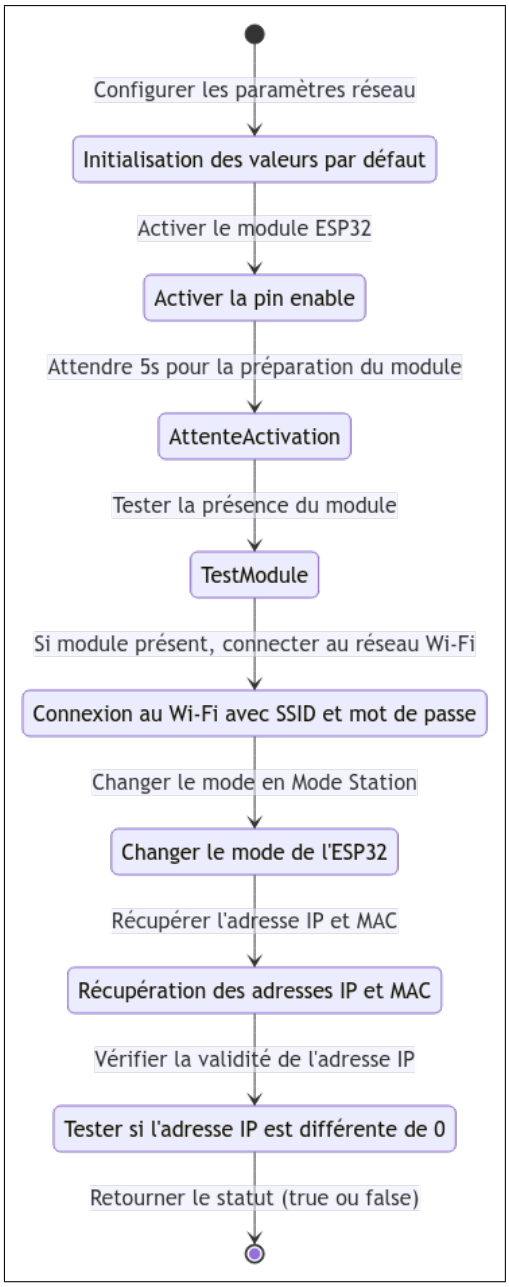


FIGURE 52 – Fonction ESP32Init

8.6.3 ESP32isStarted

Cette fonction envoie une commande AT de vérification (AT) pour tester si l'ESP32 est démarré. Ensuite, elle attend une réponse du module. Si la réponse reçue est « OK », la fonction retourne true, sinon elle retourne false.

FIGURE 53 – Fonction ESP32isStarted

8.6.4 ESP32GetIpAndMac

Cette fonction envoie une commande AT pour récupérer l'adresse IP et l'adresse MAC de l'ESP32. La fonction attend une réponse et utilise une autre fonction pour extraire ces informations de la réponse reçue.

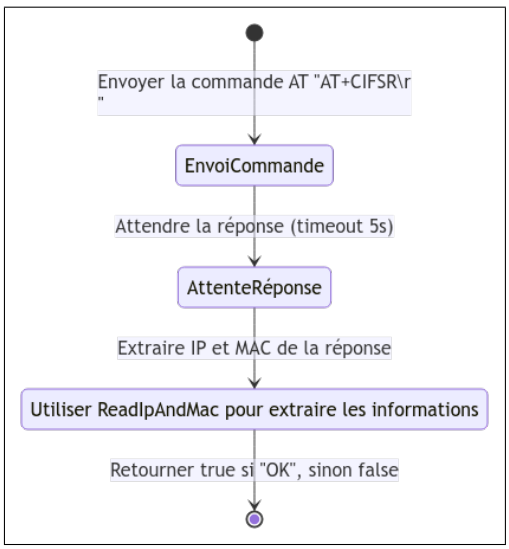


FIGURE 54 – Fonction ESP32GetIpAndMac

8.6.5 ESP32ConnectWifi

Cette fonction construit une commande AT pour connecter l'ESP32 à un réseau Wi-Fi en utilisant le SSID et le mot de passe fournis. La commande AT est ensuite envoyée au module ESP32. La fonction attend une réponse du module et retourne true si la réponse est "OK", sinon false.

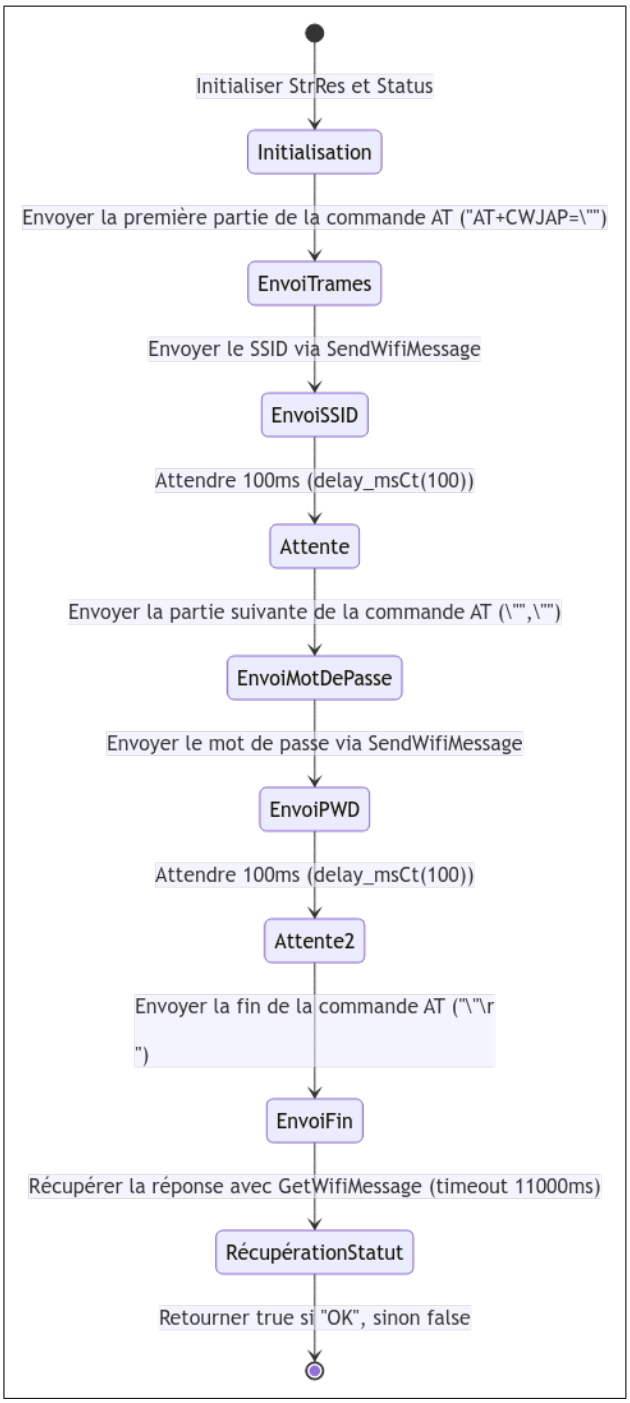


FIGURE 55 – Fonction ESP32ConnectWifi

8.6.6 ESP32ChangeMode

Cette fonction construit une commande AT pour changer le mode de l'ESP32 en fonction de la valeur fournie. La commande AT est ensuite envoyée au module ESP32. La fonction attend une réponse et retourne true si la réponse est "OK", sinon false.

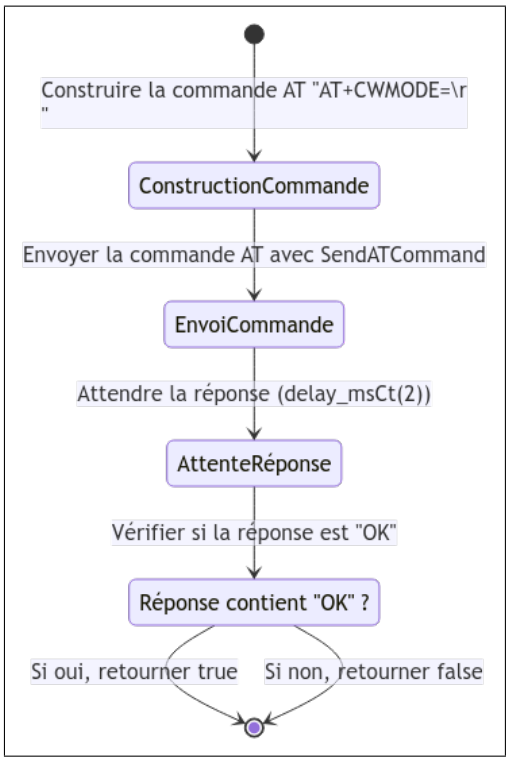


FIGURE 56 – Fonction ESP32ChangeMode

8.6.7 PublishMQTT

Cette fonction construit une commande AT pour publier un message MQTT avec le sujet et la valeur spécifiés. La commande est ensuite envoyée au module en utilisant la fonction SendATCommand.

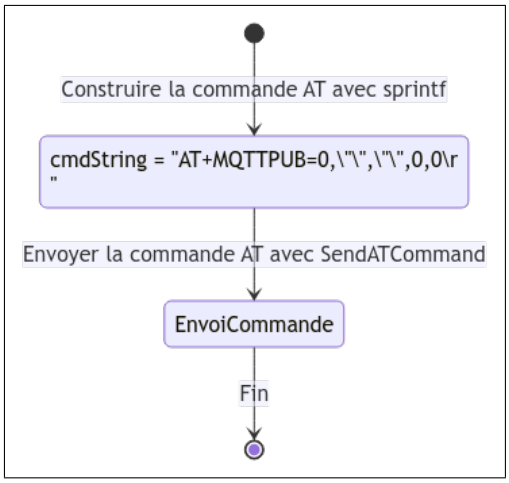


FIGURE 57 – Fonction PublishMQTT

8.6.8 SetupMQTTAndPublish

Cette fonction configure les paramètres de l'utilisateur MQTT pour ensuite se connecter à un broker MQTT pour, au final, publier les données en paramètre d'entrée sur le MQTT et, pour finir, nettoie les sessions MQTT.

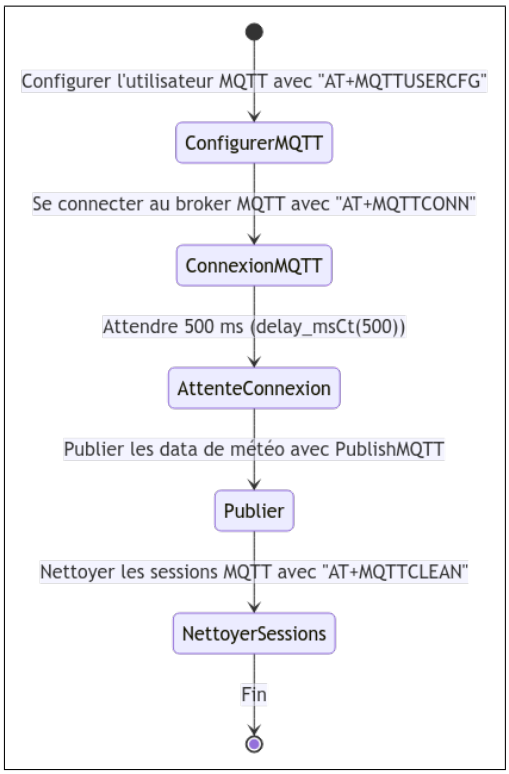


FIGURE 58 – Fonction SetupMQTTAndPublish

8.6.9 ESP32DisconnectTCPServer

Cette fonction permet de déconnecter le module Wifi au serveur auquel il est connecté.

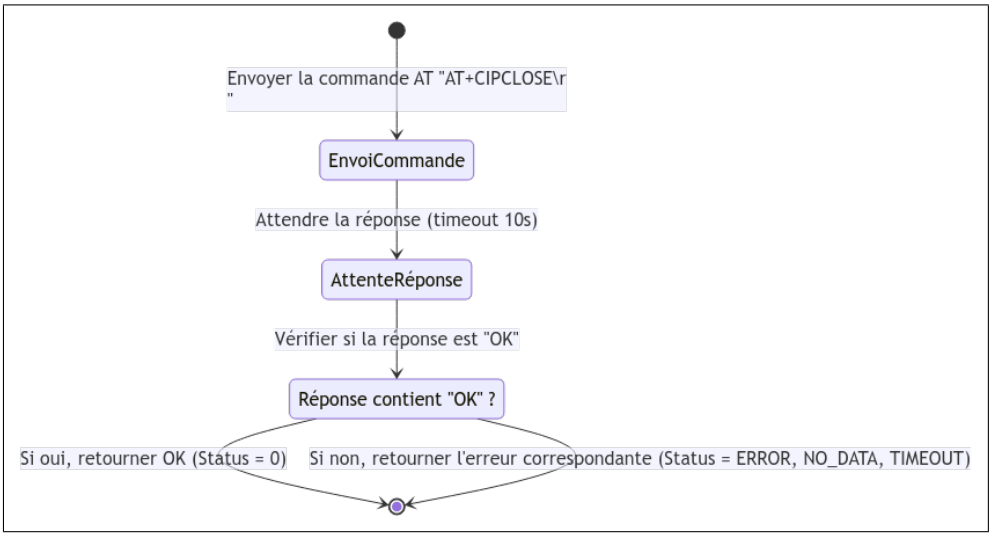


FIGURE 59 – Fonction ESP32DisconnectTCPServer

8.6.10 ESP32SendTCP

Cette fonction envoie un message test sur le serveur TCP, par la suite le but et d'envoyer une requête HTTP.

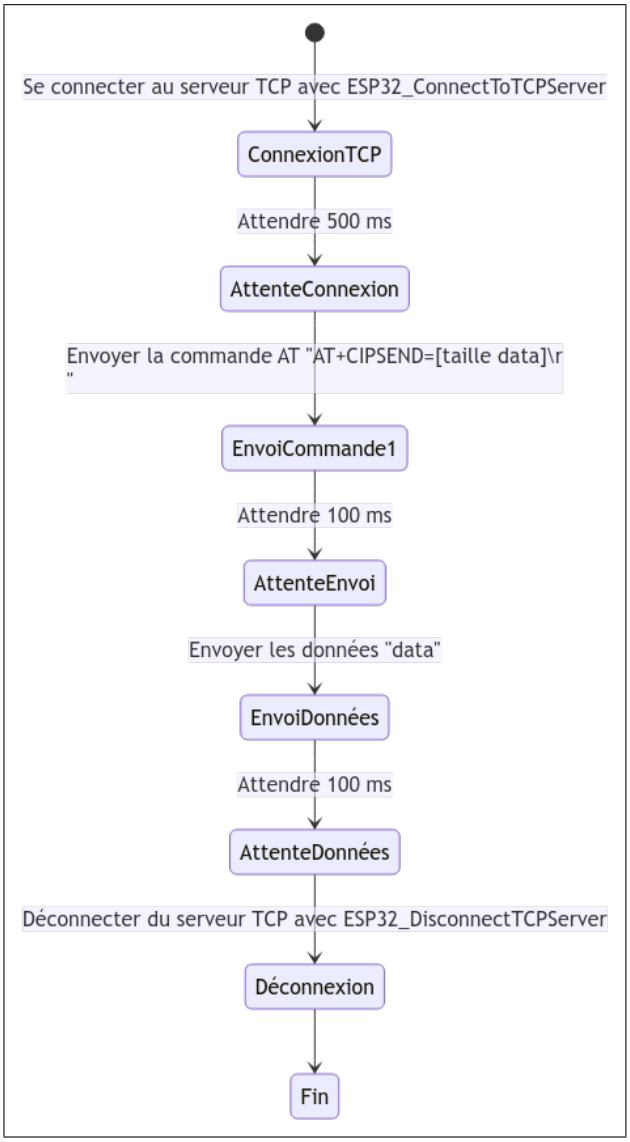


FIGURE 60 – Fonction ESP32SendTCP

8.7 Gestion menu

Pour le module wifi EPS32 je vais utiliser les fichier GestMenu.c/GestMenu.h, il dispose des fonction listé dans le tableaux 9.

Fonction	**Description**	**Ligne de début-fin**
Menuinit	Initialisation du menu	55-58
StateMenu	Machine d'etat du menu	69-393
DisplayMainMenu	Affiche le menu principal	399-420
GetUserSelection	Lecture de la sélection de l'utilisateur	426-438
MainMenuSelection	Lecture de la sélection de l'utilisateur	444-459
DisplayWifiSettings	Affiche le menu Wi-fi	465-494
WifiMenuSelection	Lecture de la sélection de l'utilisateur	500-509
DisplayMqttSettings	Affiche le menu MQTT	515-537
MqttMenuSelection	Lecture de la sélection de l'utilisateur	543-552
DisplayEcartSettings	Affiche le menu écart	557-585
EcartMenuSelection	Lecture de la sélection de l'utilisateur	591-602
MenuInput	Récupération nouvel valeur	609-623
ValueInput	Récupération nouvel valeur	630-647
DisplayDomoSettings	Affiche le menu TCP	652-674
DomoMenuSelection	Lecture de la sélection de l'utilisateur	680-689
DisplayPeriodSettings	Affiche le menu période	695-714
PeriodeMenuSelection	Lecture de la sélection de l'utilisateur	720-728
DisplayAltitudeSettings	Affiche le menu altitude	734-754
AltitudeMenuSelection	Lecture de la sélection de l'utilisateur	760-768
DisplayCurrentMeasurements	Affiche les mesure	774-806
ExitGet	Contrôle si on veut quitter le menu TCP	812-817
GestSave	Sauvgarde les données	823-842
RecupValueInSytm	Récupère les valeur du système	849-863

Table 9 – Liste des fonction de la librairie GestMenu

Cette librairie a été crée par moi est a été inspirée de la librairie menu crée par l'élève Rickenbach pour l'ancienne version du produit.

8.7.1 Menu principal

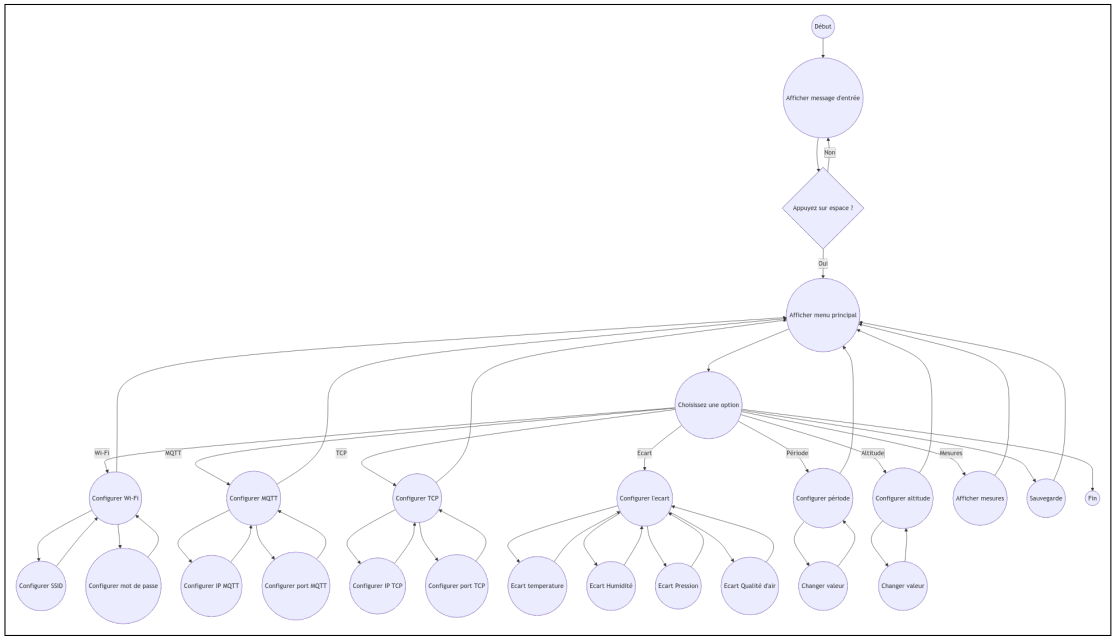


FIGURE 61 – Fonction GestMenu

Menu Principal est l'état initial où toutes les options du menu principal sont disponibles. Les transitions de Menu Principal vers d'autres états sont déclenchées par les sélections de l'utilisateur (1 à 8). Chaque option permet de revenir au Menu Principal après avoir été traitée. Le nœud Quitter termine le processus et marque la fin du diagramme avec une transition vers l'état fin.

8.8 Sauvegarde

Pour la Sauvegarde j'utileserai la librerie de l'école avec les fichier Mc32NVMUtil.c/Mc32NVMUtil.h, il dispose des fonction listé dans le tableau 10.

Fonction	**Description**	**Ligne de début-fin**
InitDataBuff	Initialisation du buffer databuff	34-40
virtualToPhysical	converti l'adresse virtuel en adresse physique	51-54
NVMpageErase	Efface page de la flash	65-102
NVMwriteRow	Écriture dans la flash	112-139
NVMArrayRead	lecture dans la flash	142-147
NVMCheckLoop	Contrôle l'état de l'opération NVM	150-161
NVMReadBlock	Ecriture d'un bloc de data dans la flash	167-186
NVMWriteBlock	lecture d'un bloc de data dans la flash	188-201
NVMResetAndWriteRow	öfface et ecrit dans la flash	203-236

TABLE 10 – Liste des fonction de la librairie Mc32NVMUtil

Des modifications en doivent être effectuées vu que cette librairie est basée sur un pic32MX795 ce qui fait que les tailles de row et de page sont différentes, pic32mx795 size row = 512 byte et size page = 4 Kbytes alors que le pic32mx130 size row = 128 byte et size page = 1 Kbytes.

8.9 RTC

Pour la RTC, je n'ai pas réussi à communiquer avec lui, mais le code a été réalisé et testé sur le kit pic32 de l'ETML-ES, et pour cela j'ai créé la librairie MCP74911 avec les fichiers MCP74911.c/MCP74911.h, il dispose des fonctions listées dans le tableau 11.

Fonction	**Description**	**Ligne de début-fin**
decToBcd	Convertisseur décimal to BCD	32-34
bcdToDec	Convertisseur BCD to décimal	44-46
MCP79411RTCWrite	ercit dans sur l'adresse donné	58-95
MCP79411RTCRead	Lecture a l'adresse donnée	107-131
MCP79411RTCInit	Init l'I2C	142-147
MCP79411RTCInitHorloge	Init l'horloge de la RTC	158-164
MCP79411RTCInitAlarm2	Init l'alarme 2 de la RTC	175-197
MCP79411RTCReadHorloge	Lecture de l'heure dans la RTC	209-218
MCP79411RTCCheckandUpdateAlarm	Contrôle l'état de l'alarme et reset	230-278

Table 11 – Liste des fonction de la librairie MCP74911

Vu que je n'ai pas put tester sur ma carte directement j'ai réalisée des test sur le kit PIC32 de l'ETML-ES.

8.10 Gestion LED RGB

Pour le moment rien de spécial n'a été relaissé pour la gestion de la LED RGB mais devrais être réalisé pour la présentation.

Pour le moment nous avons la LED qui clignote pour les mesure des capteur(LED rouge) et si l'ESP32 et connecter au Wi-fi(LED bleu) ainsi que quand la LED allumée pour un envoie donnée par l'ESP32(LED blanche).

8.11 Gestion alimentation

Pour le moment rien de spécial n'a été relaissé pour la gestion de l'alimentation mais devrais être réaliser pour la présentation.

Pour le moment j'ai pu tester les ADC avec la librairie Mc32DriverAdc de l'école avec les fichiers Mc32DriverAdc.c/Mc32DriverAdc.h, il dispose des fonctions listées dans le tableau 12.

Fonction	**Description**	**Ligne de début-fin**
BSPInitADC10	Initialisation de l'adc	32-54
BSPReadAllADC	Lecture de l'adc	60-92

Table 12 – Liste des fonction de la librairie Mc32DriverAdc

Pour pouvoir utiliser cette libraire il a fallut modifier le define de configscan et de le paramétré sur les entrées analogiques utilisé ici la valeur sera 0x600 pour les AN10 et 9.

9 Software

9.1 Introduction

Le but sera de mettre en place un serveur MQTT pour pouvoir envoyer des données depuis le produit et les récupérer ensuite. Il sera pareil pour un serveur TCP.

9.2 MQTT Broker

Pour le serveur MQTT Broker, j'ai décidé de partir de Mosquitto, auquel pas mal d'informations sont trouvables sur Internet [24].

9.2.1 Mosquitto

Pour créer mon serveur Mosquitto, je me suis aidé d'un tuto trouvé sur Internet qui est plutôt complet et compréhensible. Tout d'abord, il a fallu télécharger Mosquitto sur le site officiel mosquitto.org [25].

Ensuite, il fallut modifier les dossiers mosquitto.conf en ajoutant les lignes listener 1883. Après cela, on peut démarer Mosquitto avec l'invite de commande en mode administrateur. Après avoir ouvert l'invite de commande, on peut écrire net start mosquitto qui aura pour conséquence de démarrer Mosquitto.

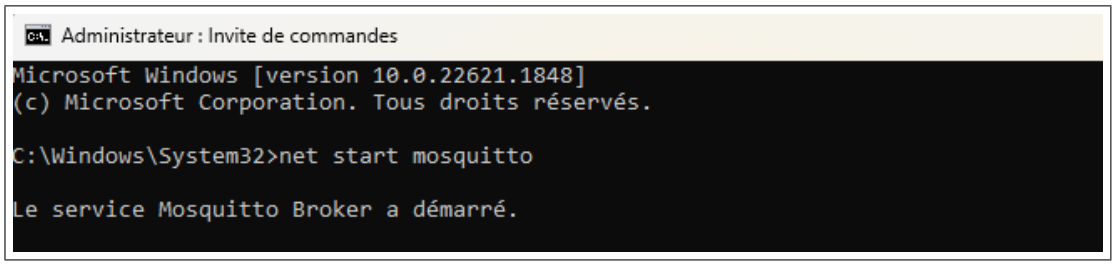


FIGURE 62 – Démarrage du serveur mosquitto

Ensuite, on peut partir sur le test pour voir si on reçoit bien la publication à laquelle nous sommes abonnés avec le même topic. Pour pouvoir tester le serveur, j'ai utilisé MQT Explorer sur mon PC portable. Pour tester, j'ai publié sur le topic sensor/CO2 450 et sur la figure 63 on peut voir la connexion au serveur et sur la figure 64 on peut voir la valeur 450 sous sensor C02.

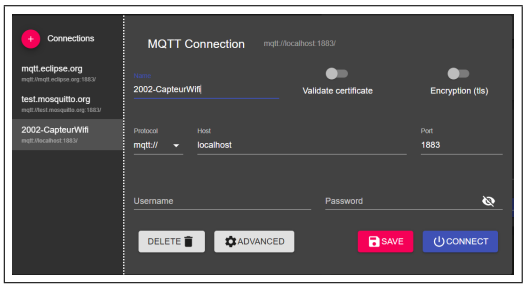


FIGURE 63 – connexion au serveur mosquitto sur mqttexplorer

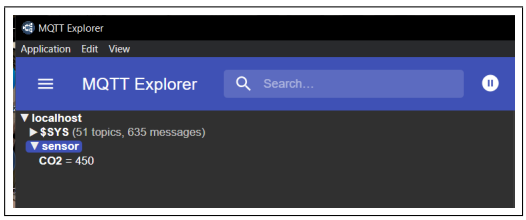


FIGURE 64 – réception de donnée sur mqttexplorer

Maintenant, nous savons que le serveur est fonctionnel et qu'il peut être utilisé plus tard pour les tests avec le produit final.

24. Tuto MQTT sur ESP32 + Mosquittto
25. Site officiel

J'ai également fait le test avec une application mobile pour pouvoir voir les data depuis mon téléphone. L'application s'appelle myMQTT. Sur la figure65 on peut voir les paramètres de connexion et sur la figure66 Le dashboard, ici aussi, j'ai testé avec l'envoi de 450 sur le topic sensor/CO2.

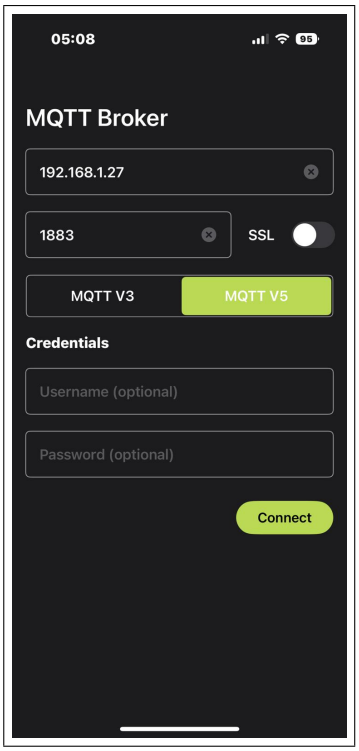


FIGURE 65 – Connexion application myMQTT

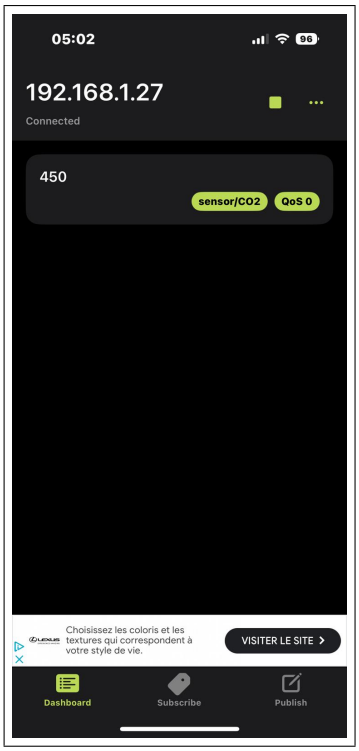


FIGURE 66 – Dashboard de l'application

9.3 TCP

Pour la communication avec un serveur TCP j'ai utilisé une application mobile qui crée un serveur sur le téléphone et on peut également visualiser les données envoyées sur le serveur. L'application s'appelle TCP UDP S-C.

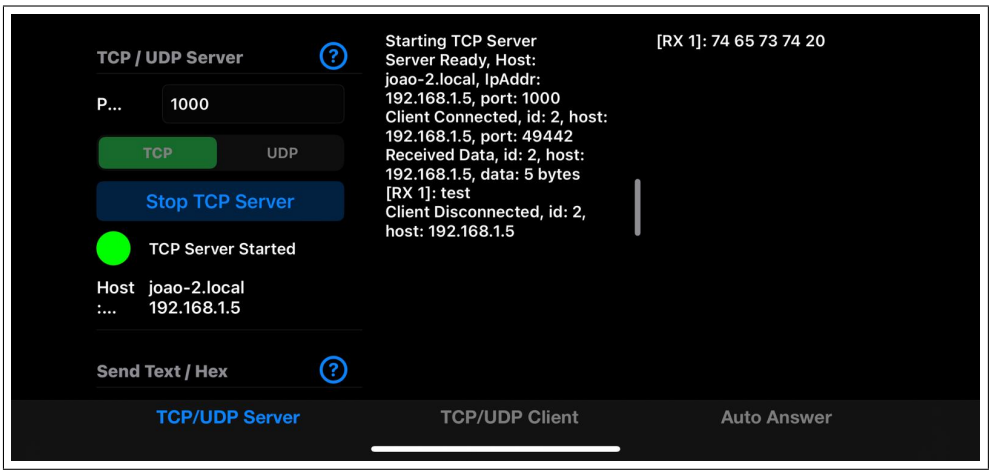


FIGURE 67 – Test application serveur TCP

Sur la figure 67 on peut voir le serveur joao-2.local qui a bien reçu le message test <test>.

10 Mesures

Les mesures visent à prouver le bon fonctionnement du système et à vérifier que le cahier des charges est respecté.

10.1 Matériel utilisé

Oscilloscope	Rhode and Schwarz	ES.SLO2.05.01.11
Multimètre	Gwinstek	R110-12

TABLE 13 – Liste du matériel de mesure

10.2 I2C

10.2.1 BME280

Pour cette mesure, on a utilisé les points de test SENS SDA et SENS SCL.

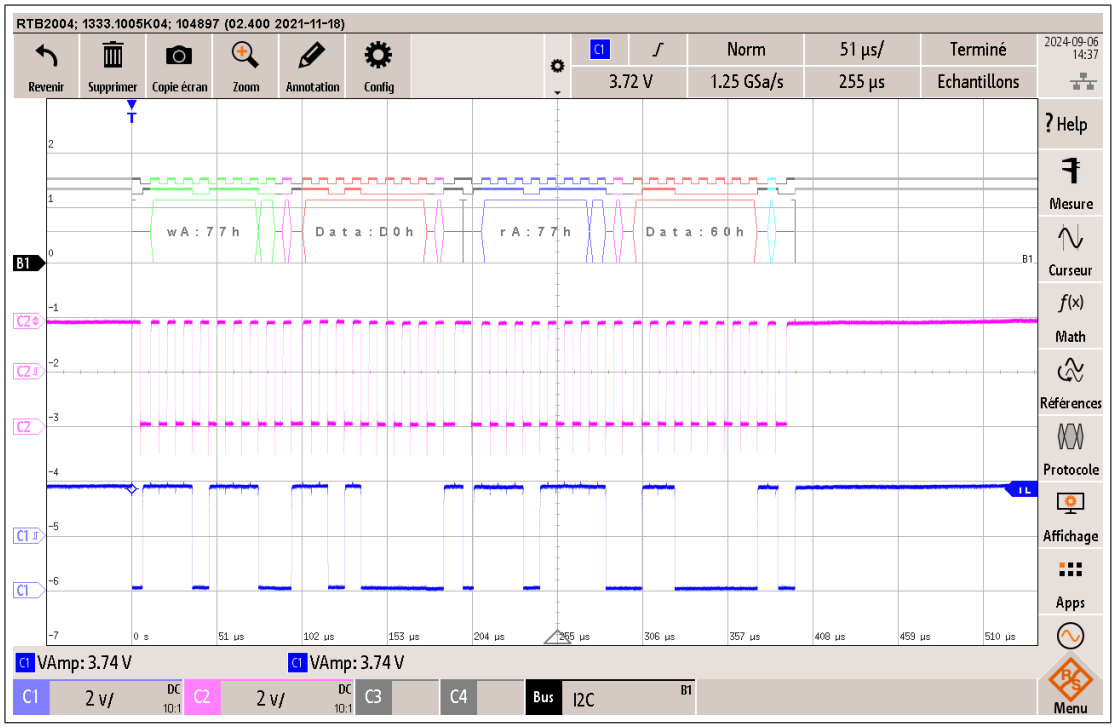


FIGURE 68 – mesure i2c du BME280

Sur la figure 68 on peut voir que nous avons bien une communication avec le BME280 et nous pouvons également voir que nous recevons son ID qui est de 60, comme l'indique le datasheet, comme on peut le voir sur la figure 69.

5.4 Register description

5.4.1 Register 0xD0 *"id"*

The *"id"* register contains the chip identification number chip_id[7:0], which is 0x60. This number can be read as soon as the device finished the power-on-reset.

FIGURE 69 – ID du BME280

10.2.2 ENS160

Pour cette mesure, on a utilisé les points de test SENS SDA et SENS SCL.

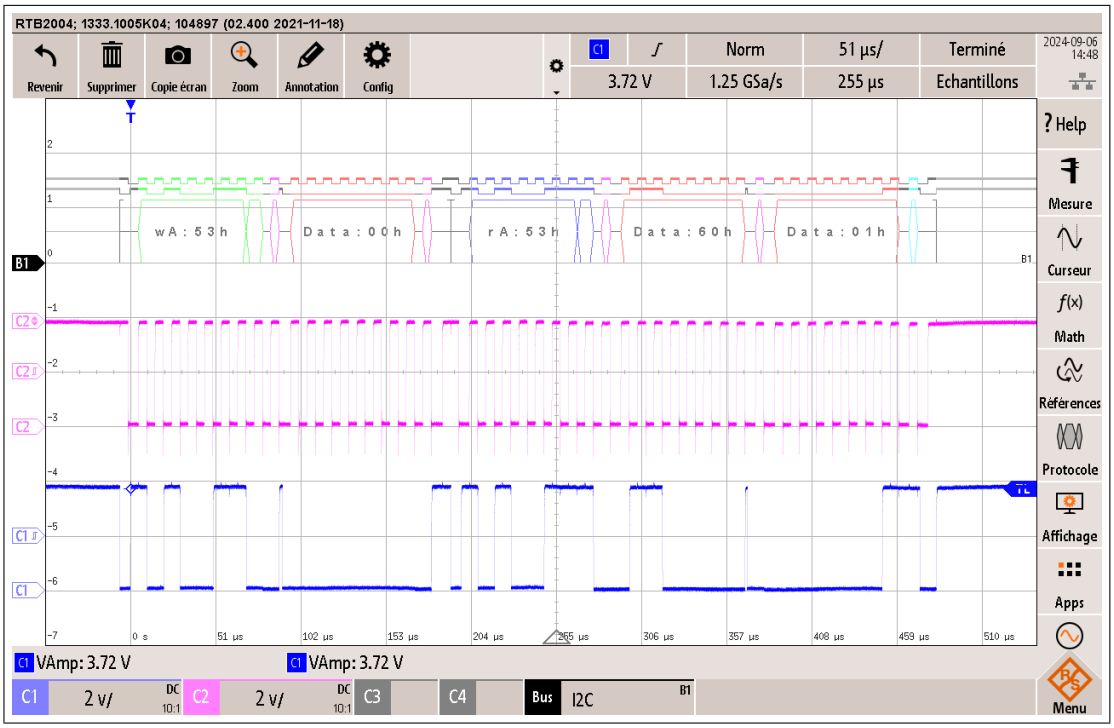


FIGURE 70 – mesure i2c du ENS160

Sur la figure 70 on peut voir que nous avons bien une communication avec le ENS160 et nous pouvons également voir que nous recevons son ID qui est de 60 01, comme l'indique le datasheet, comme on peut le voir sur la figure 71.

16.2.1 PART_ID (Address 0x00)

This 2-byte register contains the part number in little endian of the ENS160.

The value is available when the ENS160 is initialized after power-up.

Table 17: Register PART_ID

Address 0x00		PART_ID		
Bits	Field Name	Default	Access	Field Description
0:7	PART_ID_LSB	0x60	read	Lower Byte of Part ID
8:15	PART_ID_MSB	0x01	read	Upper Byte of Part ID

FIGURE 71 – ID du ENS160

10.2.3 MCP74911

Pour cette mesure, on a utilisé les points de test RTC SDA et RTC SCL.

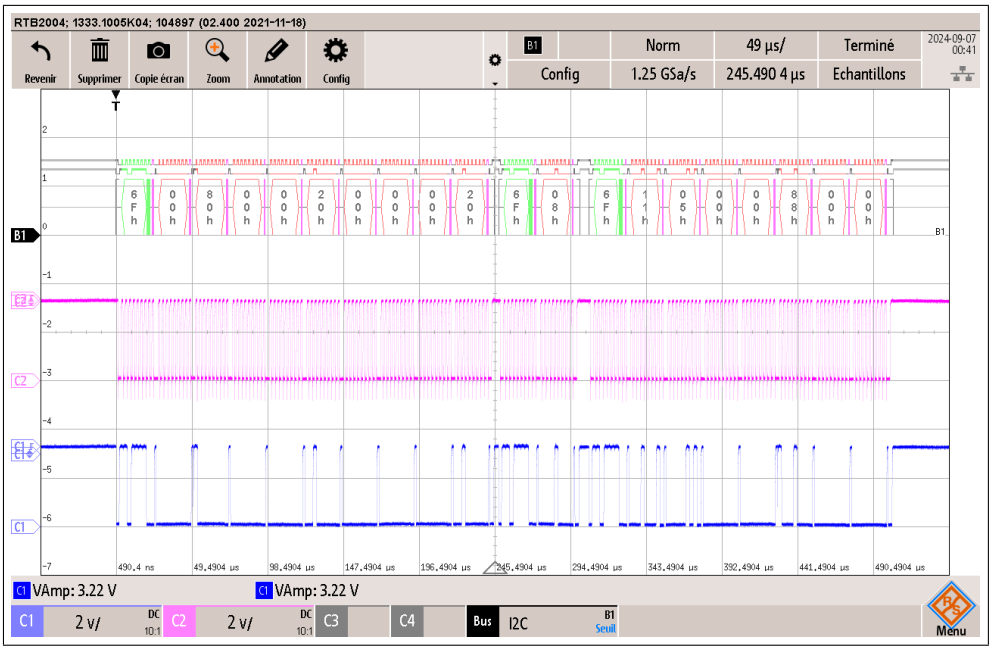


FIGURE 72 – Configuration du MCP74911

Sur la figure 72 on peut voir la configuration du MCP74911 qui est bien transmise en I2C et sur la figure 73 on peut voir le résultat de cette configuration qui est l'alarme qui s'active et à laquelle le code reset le flag pour toutes les 5 s.

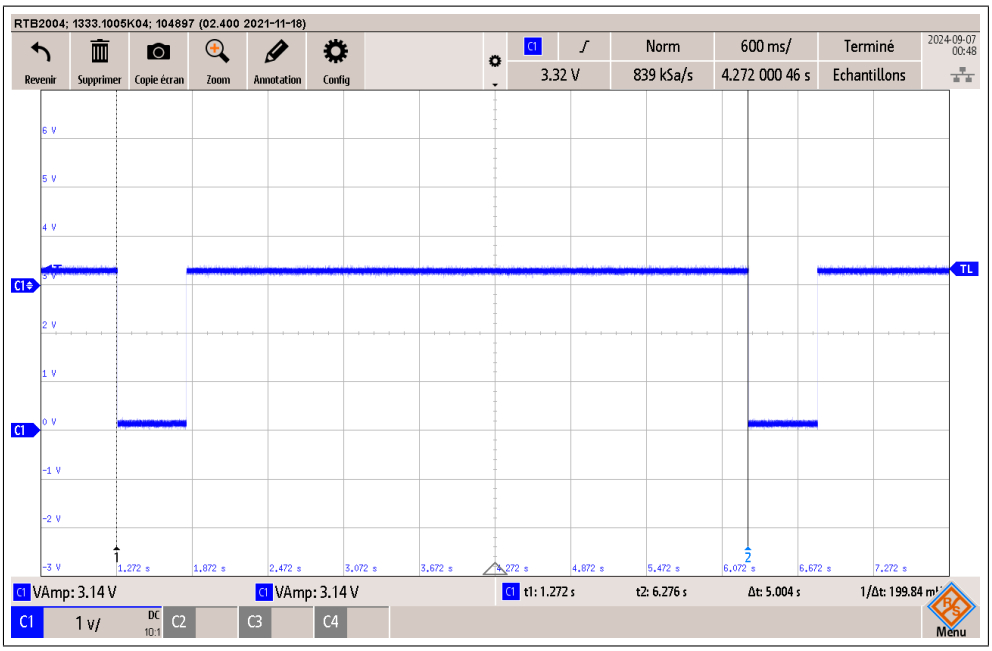


FIGURE 73 – Alarme toutes les 5s

10.3 Uart

10.3.1 ESP32

Pour cette mesure, on a utilisé les points de test ESPrx et ESPtx.

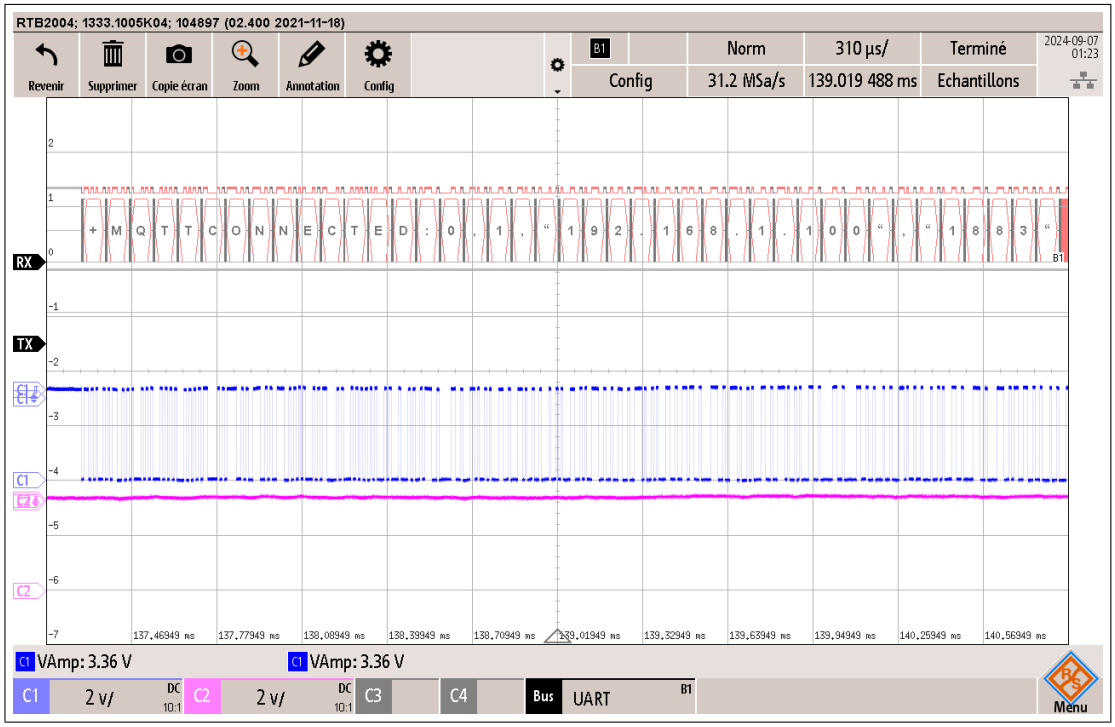


FIGURE 74 – réponse de l'ESP32

Sur la figure 74 on peut la reposer à la commande de connexion au serveur MQTT et ici, on voit qu'il répond qu'il est connecté.

10.3.2 FT230X

Pour cette mesure, on a utilisé les points de test USBrx et USBtx.

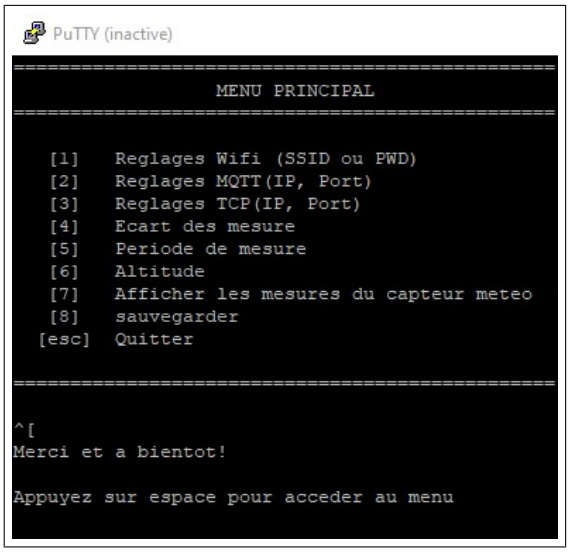


FIGURE 75 – Réception sur Putty

Sur la figure 75 on peut voir ce que reçoit Putty à travers du FT230X et sur la figure76 on peut voir la trame uart du message. Merci à bientôt.

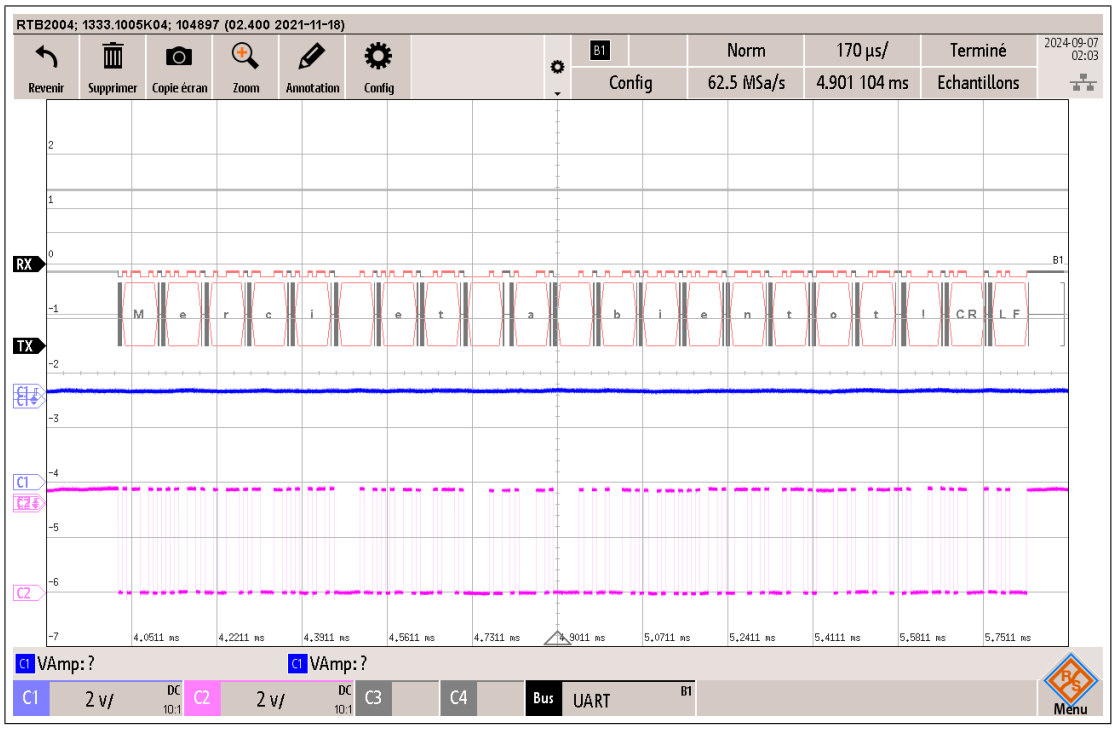


FIGURE 76 – trame UART de merci a bientôt

10.4 Alimentation

On peut voir dans le tableau 14 que les tensions d'alimentation sont correctes et qu'il n'y a pas de problème. Pour cette mesure, on a utilisé les points de test Vusb et le 3V3.

VUSB	5 V	5.05 V
VOUT	3.3 V	3.295 V

Table 14 – Mesure DC

10.5 Lecture ADC

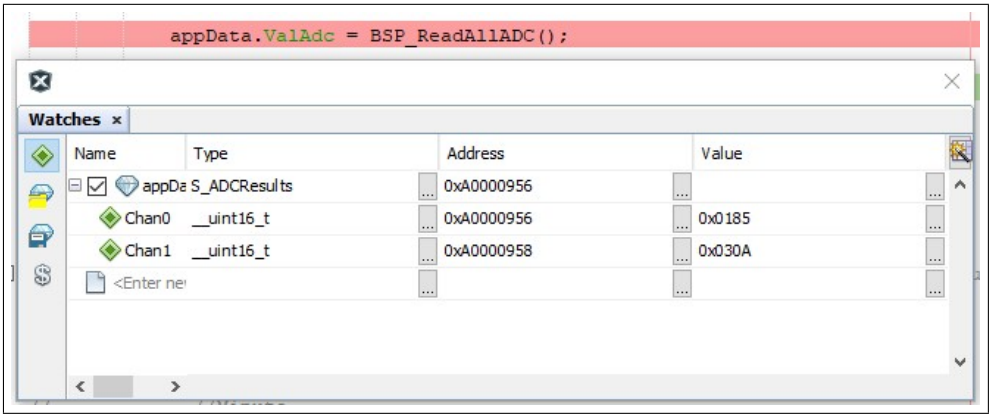


Figure 77 – Lecture ADC

On peut voir sur la figure77 que nous lisons bien quelque chose maintenant. Il faut les convertir et comparer avec les mesures effectuées sur la cellule 4 et 3. Pour convertir, il suffit de faire * 3300/1024.

Celule 4	1.256 V	1.254
Celule 3	2.521 V	2.507

Table 15 – Mesure ADC

On peut voir dans le tableau 15 que les valeurs sont proches et que notre lecture ADC est fonctionnelle.

11 Test mémoire Flash

```
//saugarde flash
NVM_ReadBlock((uint32_t *)&Struct_save,sizeof(S_SAVE));
Struct_save.Magic = 0x12345678;
NVM_WriteBlock((uint32_t *)&Struct_save,sizeof(S_SAVE));
```

Watches

Name	Type	Address	Value
databuff[5]	__uint32_t	0xA00011D4	0x12345678
eedata_addr[5]	__uint32_t	0x9D005C14	0x00000000
eedata_addr	__uint32_t[32]	0x9D005C00	
databuff	__uint32_t[32]	0xA00011C0	
NVMSRCADDR	SFR	0x1F80F440	0x000011C0
NVMADDR	SFR	0x1F80F420	0x1D005C00

FIGURE 78 – Problème de flash

Avec la figure78 on peut voir que les adresses des tableaux sont correctes(page 47 du datasheet du pic32MX130 [26]), mais que le transfert entre databuff et eedata ne se fait pas. Il est possible que la mémoire soit bloquée pour une quelconque raison, mais des recherches seront faites après le rendu du rapport.

26. Fiche technique du PIC32MX130F256D

12 Auto-analyse

12.1 Acquis du TD

Ce projet a été une bonne opportunité pour mettre en place la pratique et les compétences apprises en cours. J'ai pu travailler avec des microcontrôleurs, des capteurs, et pour la première fois, j'ai travaillé sur le module ESP32 pour la connectivité Wi-Fi. Ces technologies ont été essentielles pour transmettre les données. Du début à la fin, dans chaque étape, on pouvait appliquer nos connaissances en électronique, en programmation. En plus des aspects techniques de base, ce projet nous a aussi permis de nous plonger dans des concepts plus avancés comme la gestion de l'énergie et l'optimisation des performances, renforçant ainsi ce que nous avons appris en théorie. Maîtriser les protocoles de communication et savoir intégrer divers composants dans un système harmonieux ont été des leçons clés qui nous ouvrent de nombreuses portes pour des applications futures dans les systèmes embarqués.

12.2 Prise en compte de l'environnement social

Le projet prend en compte des aspects sociaux en facilitant l'accès aux données environnementales via une interface Web. Cette transparence dans la mise à disposition des informations peut sensibiliser les utilisateurs aux conditions climatiques locales et à la qualité de l'air, contribuant ainsi à une meilleure prise de conscience des enjeux environnementaux. En effet, en rendant les données facilement accessibles et compréhensibles, le projet encourage un comportement plus proactif de la part des utilisateurs concernant leur environnement immédiat. De plus, le partage des données via des protocoles standardisés comme MQTT encourage la communauté à intégrer ces informations dans des systèmes domotiques, des applications de santé publique, ou même des projets de recherche plus larges. Cela permet également de créer des synergies entre différentes plateformes IoT, favorisant l'innovation collaborative.

12.3 Prise en compte de l'environnement naturel

Le capteur ENS160, utilisé dans le projet, permet de mesurer la qualité de l'air en fournissant des informations cruciales sur la concentration de CO2 et de TVOC. Ces données peuvent être utilisées pour évaluer et améliorer la qualité de l'air intérieur, impactant ainsi positivement la santé et le bien-être des utilisateurs. En outre, en intégrant des modules à faible consommation d'énergie et en prévoyant des sources d'alimentation de secours, le projet vise à réduire son empreinte écologique tout en assurant une opération continue. L'optimisation de la consommation énergétique et la gestion efficace des ressources sont des considérations clés qui montrent la prise en compte de l'impact environnemental à chaque étape du développement.

12.4 Leadership et développement personnel

Le projet a favorisé le développement personnel en exigeant un haut niveau d'autonomie, une gestion rigoureuse du temps, et une capacité accrue à résoudre des problèmes complexes. Travailler sur ce projet a également amélioré les compétences en communication et en travail d'équipe, en particulier lors de la collaboration avec des mentors, des pairs, et d'autres experts du domaine. Le leadership a été démontré à travers la prise d'initiatives pour résoudre des défis techniques, tels que le dépannage des capteurs, l'optimisation des performances du module ESP32, et la gestion efficace des ressources du projet. Ce projet a également offert des opportunités de développer des compétences en gestion de projet, telles que la planification, la gestion des risques, et l'itération rapide en réponse aux tests et aux retours d'expérience. Enfin, le travail réalisé a renforcé la confiance en soi dans la capacité à mener à bien des projets techniques complexes, et a montré l'importance de la persévérance et de l'apprentissage continu dans le domaine des technologies embarquées et de l'IoT.

13 Conclusion

Le projet de capteur météo Wi-Fi est presque fini, atteignant ses objectifs principaux d'intégration et de communication de données environnementales via un module ESP32. Ce projet visait à créer un dispositif capable de mesurer divers paramètres climatiques tels que la pression, l'humidité, la température, et la qualité de l'air. L'utilisation de capteurs tels que le BME280 et l'ENS160 a permis d'obtenir des mesures précises, qui sont ensuite transmises à un serveur MQTT pour être accessibles via une interface Web. Cette infrastructure offre une solution pratique pour la surveillance continue des conditions environnementales et peut être facilement intégrée dans des systèmes domotiques existants ou des plateformes de recherche.

Bien que le système ait été fonctionnel, certaines améliorations ont été identifiées pour les futures itérations. La gestion de l'alimentation pourrait être optimisée pour prolonger l'autonomie en mode batterie ainsi que trouver la source du probleme du chargeur NiMH. De plus, la performance du module MQTT et les délais de réponse du réseau ont été satisfaisants, mais une évaluation continue serait bénéfique pour identifier d'éventuels goulots d'étranglement lors d'une montée en charge avec plusieurs dispositifs connectés simultanément.

Ce projet a été une expérience riche en enseignements, mettant en évidence l'importance de la planification minutieuse et de la flexibilité dans la gestion des projets techniques complexes. Le travail a renforcé l'importance de la documentation, non seulement pour le développement immédiat, mais aussi pour faciliter la maintenance et les améliorations futures. La collaboration avec les mentors et les pairs a été cruciale pour surmonter les défis et pour pousser le projet au-delà de ses limites initiales.

En somme, ce projet de capteur météo Wi-Fi a non seulement permis de concrétiser les compétences acquises durant le cursus académique, mais il a également posé des bases solides pour des développements futurs dans le domaine des systèmes de surveillance environnementale connectés. Le dispositif développé présente un potentiel d'application vaste, et ses améliorations futures pourraient encore augmenter sa valeur et son impact.

Joao Marques
ETML-ES
Lausanne,25.03.2024

14 bibliographie

https ://cdn.sparkfun.com/assets/learn$_{t}utorials/4/1/9/BST - BME280_{D}S001 - 10.pdf$

https ://cdn.sparkfun.com/assets/3/c/7/5/5/SC-001224-DS-7-ENS160-Datasheet.pdf

https ://www.mouser.ch/datasheet/2/268/MCP79410$_{M}CP79411_{M}CP79412_{B}attery_{B}acked_{I}2C_{R}TCC-$ $3359732.pdf$

https ://www.mouser.ch/datasheet/2/216/APHF1608LSEEQBDZGKC-1145215.pdf

https ://fscdn.rohm.com/en/products/databook/datasheet/opto/led/chip$_{m}ono/sml - p11vtt86(r) - e.pdf$

https ://www.mouser.ch/datasheet/2/268/PIC32MX1XX2XX283644$_{P}IN_{D}atasheet_{D}S60001168L - 3443206.pdf$

https ://www.ti.com/lit/ds/symlink/bq25172.pdf

https ://www.we-online.com/components/products/datasheet/173950336.pdf

https ://www.procell.com/wp-content/uploads/2023/04/Datasheet$_{C}onstant_{A}AA -_{P} C2400.pdf$

https ://www.mouser.ch/datasheet/2/315/3513$_{f}ileversion - 1627229.pdf$

https ://www.mouser.ch/datasheet/2/258/SI5618$_{S}OT_{2}3_2510086.pdf$

https ://docs.espressif.com/projects/esp-at/en/latest/esp32c3/index.html

https ://www.espressif.com/sites/default/files/documentation/esp32-c3-wroom-02$_{d}atasheet_{e}n.pdf$

https ://ftdichip.com/wp-content/uploads/2023/07/DS$_{T}TL - 232R_{C}ABLES.pdf$

https ://docs.espressif.com/projects/esp-at/en/latest/esp32c3/AT$_{B}inary_{L}ists/esp_{a}t_{b}inaries.html$

https ://www.espressif.com/en/support/download/other-tools

https ://letmeknow.fr/fr/blog/180-tuto-mqtt-sur-esp32–mosquitto

https ://mosquitto.org

15 Annexe

15.1 Cahier des charges

15.2 Planification

15.3 Journal de travail

15.4 Procès-verbaux des séances hebdomadaires

15.5 Schémas électroniques

15.6 Fichiers de fabrication

15.7 Code source firmware